

Ayurvedic Book Processor - Line By Line Code Explanation

Generated: 2026-05-03 21:19

Purpose: explain the production code in plain English for coworkers who need to understand what the app does without first reading Python.

Security note: this document explains .env.example only. It does not print the real .env values or the paid Gemini API key.

How To Read This

Each source line is shown with its file line number. The explanation tells what that line does inside the app. Blank lines are included because this is a line-by-line walkthrough.

app.py

Web application, upload UI, queue, auth, downloads, and background job runner.

Source path: C:\Users\sawan\Desktop\new_project\Ayurvedic_Book_Processor\app.py

Line count: 792

Line 1: from __future__ import annotations

Explanation: Enables future Python behavior for type annotations.

Line 2: [blank]

Explanation: Blank line used to separate logical sections.

Line 3: import json

Explanation: Imports a Python module used later in this file.

Line 4: import secrets

Explanation: Imports a Python module used later in this file.

Line 5: import os

Explanation: Imports a Python module used later in this file.

Line 6: import shutil

Explanation: Imports a Python module used later in this file.

Line 7: import subprocess

Explanation: Imports a Python module used later in this file.

Line 8: import sys

Explanation: Imports a Python module used later in this file.

Line 9: import threading

Explanation: Imports a Python module used later in this file.

Line 10: import uuid

Explanation: Imports a Python module used later in this file.

Line 11: from datetime import datetime

Explanation: Imports specific classes or functions from another module.

Line 12: from pathlib import Path

Explanation: Imports specific classes or functions from another module.

Line 13: from typing import Any

Explanation: Imports specific classes or functions from another module.

Line 14: from html import escape

Explanation: Imports specific classes or functions from another module.

Line 15: [blank]

Explanation: Blank line used to separate logical sections.

Line 16: from flask import Flask, Response, jsonify, redirect, request, send_file, url_for

Explanation: Imports specific classes or functions from another module.

Line 17: from dotenv import load_dotenv

Explanation: Imports specific classes or functions from another module.

Line 18: from werkzeug.exceptions import RequestEntityTooLarge

Explanation: Imports specific classes or functions from another module.

Line 19: [blank]

Explanation: Blank line used to separate logical sections.

Line 20: [blank]

Explanation: Blank line used to separate logical sections.

Line 21: ROOT = Path(__file__).resolve().parent

Explanation: Assigns or updates the value of ROOT.

Line 22: JOBS_DIR = ROOT / "jobs"

Explanation: Assigns or updates the value of JOBS_DIR.

Line 23: COMPLETED_DOCS_DIR = ROOT / "completed_docs"

Explanation: Assigns or updates the value of COMPLETED_DOCS_DIR.

Line 24: LOG_DIR = ROOT / "logs"

Explanation: Assigns or updates the value of LOG_DIR.

Line 25: ENV_FILE = ROOT / ".env"

Explanation: Assigns or updates the value of ENV_FILE.

Line 26: ENV_EXAMPLE = ROOT / ".env.example"

Explanation: Assigns or updates the value of ENV_EXAMPLE.

Line 27: PROCESSOR = ROOT / "ultimate_book_processor.py"

Explanation: Assigns or updates the value of PROCESSOR.

Line 28: JOBS_DB = JOBS_DIR / "jobs.json"

Explanation: Assigns or updates the value of JOBS_DB.

Line 29: [blank]

Explanation: Blank line used to separate logical sections.

Line 30: load_dotenv(ENV_FILE)

Explanation: Continues the logic inside this file.

Line 31: [blank]

Explanation: Blank line used to separate logical sections.

Line 32: app = Flask(__name__)

Explanation: Assigns or updates the value of app.

Line 33: app.config["MAX_CONTENT_LENGTH"] = int(os.getenv("MAX_UPLOAD_MB", "250")) * 1024 * 1024

Explanation: Assigns or updates the value of app.config["MAX_CONTENT_LENGTH"].

Line 34: [blank]

Explanation: Blank line used to separate logical sections.

Line 35: jobs_lock = threading.Lock()

Explanation: Assigns or updates the value of jobs_lock.

Line 36: dispatcher_lock = threading.Lock()

Explanation: Assigns or updates the value of dispatcher_lock.

Line 37: running_processes: dict[str, subprocess.Popen[str]] = {}

Explanation: Starts the processor as a separate background process for the job.

Line 38: dispatcher_started = False

Explanation: Assigns or updates the value of dispatcher_started.

Line 39: [blank]

Explanation: Blank line used to separate logical sections.

Line 40: [blank]

Explanation: Blank line used to separate logical sections.

Line 41: `def now_text() -> str:`

Explanation: Returns the current local timestamp for job status fields.

Line 42: `return datetime.now().isoformat(timespec="seconds")`

Explanation: Returns the result from `now_text`.

Line 43: [blank]

Explanation: Blank line used to separate logical sections.

Line 44: [blank]

Explanation: Blank line used to separate logical sections.

Line 45: `def ensure_project_files() -> None:`

Explanation: Creates required project folders and starter files if they are missing.

Line 46: `JOBS_DIR.mkdir(parents=True, exist_ok=True)`

Explanation: Creates a folder if it does not already exist.

Line 47: `COMPLETED_DOCS_DIR.mkdir(parents=True, exist_ok=True)`

Explanation: Creates a folder if it does not already exist.

Line 48: `LOG_DIR.mkdir(parents=True, exist_ok=True)`

Explanation: Creates a folder if it does not already exist.

Line 49: `if not ENV_FILE.exists() and ENV_EXAMPLE.exists():`

Explanation: Starts a conditional branch; the indented block runs only when this condition is true.

Line 50: `ENV_FILE.write_text(ENV_EXAMPLE.read_text(encoding="utf-8-sig"), encoding="utf-8")`

Explanation: Writes text content to a file.

Line 51: `if not JOBS_DB.exists():`

Explanation: Starts a conditional branch; the indented block runs only when this condition is true.

Line 52: `save_jobs([])`

Explanation: Continues the logic inside `ensure_project_files`.

Line 53: [blank]

Explanation: Blank line used to separate logical sections.

Line 54: [blank]

Explanation: Blank line used to separate logical sections.

Line 55: `def valid_job_id(job_id: str) -> bool:`

Explanation: Checks that a job id has the expected timestamp plus random suffix format.

Line 56: `if len(job_id) != 24:`

Explanation: Starts a conditional branch; the indented block runs only when this condition is true.

Line 57: `return False`

Explanation: Returns the result from `valid_job_id`.

Line 58: `prefix, suffix = job_id[:15], job_id[16:]`

Explanation: Continues the logic inside `valid_job_id`.

Line 59: `return job_id[15] == "_" and prefix.replace("_", "").isdigit() and suffix.isalnum()`

Explanation: Returns the result from `valid_job_id`.

Line 60: [blank]

Explanation: Blank line used to separate logical sections.

Line 61: [blank]

Explanation: Blank line used to separate logical sections.

Line 62: `def read_env(path: Path = ENV_FILE) -> dict[str, str]:`

Explanation: Reads key/value settings from a .env-style file.

Line 63: `ensure_project_files()`

Explanation: Continues the logic inside `read_env`.

Line 64: values: dict[str, str] = {}

Explanation: Assigns or updates the value of values: dict[str, str].

Line 65: if not path.exists():

Explanation: Starts a conditional branch; the indented block runs only when this condition is true.

Line 66: return values

Explanation: Returns the result from read_env.

Line 67: for line in path.read_text(encoding="utf-8-sig").splitlines():

Explanation: Starts a loop over each item in a collection.

Line 68: if not line or line.lstrip().startswith("#") or "=" not in line:

Explanation: Starts a conditional branch; the indented block runs only when this condition is true.

Line 69: continue

Explanation: Continues the logic inside read_env.

Line 70: key, value = line.split("=", 1)

Explanation: Continues the logic inside read_env.

Line 71: values[key.strip()] = value.strip()

Explanation: Assigns or updates the value of values[key.strip()].

Line 72: return values

Explanation: Returns the result from read_env.

Line 73: [blank]

Explanation: Blank line used to separate logical sections.

Line 74: [blank]

Explanation: Blank line used to separate logical sections.

Line 75: def write_env(updates: dict[str, str]) -> None:

Explanation: Writes updated settings back to .env in a stable order.

Line 76: ensure_project_files()

Explanation: Continues the logic inside write_env.

Line 77: existing = read_env()

Explanation: Assigns or updates the value of existing.

Line 78: existing.update(updates)

Explanation: Continues the logic inside write_env.

Line 79: ordered_keys = [

Explanation: Assigns or updates the value of ordered_keys.

Line 80: "GEMINI_API_KEY",

Explanation: Continues the logic inside write_env.

Line 81: "GEMINI_MODEL",

Explanation: Continues the logic inside write_env.

Line 82: "PDF_DIR",

Explanation: Continues the logic inside write_env.

Line 83: "OUTPUT_DIR",

Explanation: Continues the logic inside write_env.

Line 84: "POPPLER_PATH",

Explanation: Continues the logic inside write_env.

Line 85: "DPI",

Explanation: Continues the logic inside write_env.

Line 86: "MAX_RETRIES",

Explanation: Continues the logic inside write_env.

Line 87: "GEMINI_DELAY_SECONDS",

Explanation: Continues the logic inside write_env.

Line 88: "GEMINI_TIMEOUT_SECONDS",

Explanation: Continues the logic inside write_env.

Line 89: "SPEED_MODE",

Explanation: Continues the logic inside write_env.

Line 90: "PAGE_WORKERS_PER_JOB",

Explanation: Continues the logic inside write_env.

Line 91: "RESET_MASTER",

Explanation: Continues the logic inside write_env.

Line 92: "CREATE_DOCX",

Explanation: Continues the logic inside write_env.

Line 93: "TEST_START_PAGE",

Explanation: Continues the logic inside write_env.

Line 94: "TEST_MAX_PAGES",

Explanation: Continues the logic inside write_env.

Line 95: "TEST_PDF_LIMIT",

Explanation: Continues the logic inside write_env.

Line 96: "CREATE_STRUCTURED_NOTES",

Explanation: Continues the logic inside write_env.

Line 97: "MAX_PARALLEL_JOBS",

Explanation: Continues the logic inside write_env.

Line 98: "MAX_UPLOAD_MB",

Explanation: Continues the logic inside write_env.

Line 99: "APP_USERNAME",

Explanation: Continues the logic inside write_env.

Line 100: "APP_PASSWORD",

Explanation: Continues the logic inside write_env.

Line 101:]

Explanation: Continues the logic inside write_env.

Line 102: lines: list[str] = []

Explanation: Assigns or updates the value of lines: list[str].

Line 103: for key in ordered_keys:

Explanation: Starts a loop over each item in a collection.

Line 104: if key in existing:

Explanation: Starts a conditional branch; the indented block runs only when this condition is true.

Line 105: lines.append(f"{key}={existing[key]}")

Explanation: Assigns or updates the value of lines.append(f"{key}").

Line 106: for key in sorted(set(existing) - set(ordered_keys)):

Explanation: Starts a loop over each item in a collection.

Line 107: lines.append(f"{key}={existing[key]}")

Explanation: Assigns or updates the value of lines.append(f"{key}").

Line 108: ENV_FILE.write_text("\n".join(lines) + "\n", encoding="utf-8")

Explanation: Writes text content to a file.

Line 109: load_dotenv(ENV_FILE, override=True)

Explanation: Continues the logic inside write_env.

Line 110: upload_mb = existing.get("MAX_UPLOAD_MB", os.getenv("MAX_UPLOAD_MB", "250"))

Explanation: Assigns or updates the value of upload_mb.

Line 111: app.config["MAX_CONTENT_LENGTH"] = int(upload_mb) * 1024 * 1024

Explanation: Assigns or updates the value of app.config["MAX_CONTENT_LENGTH"].

Line 112: [blank]

Explanation: Blank line used to separate logical sections.

Line 113: [blank]

Explanation: Blank line used to separate logical sections.

Line 114: `def max_parallel_jobs() -> int:`

Explanation: Reads and clamps how many background jobs may run at the same time.

Line 115: `try:`

Explanation: Starts a protected block where errors can be caught.

Line 116: `return max(1, min(10, int(read_env().get("MAX_PARALLEL_JOBS", "3"))))`

Explanation: Returns the result from `max_parallel_jobs`.

Line 117: `except ValueError:`

Explanation: Handles an error raised inside the matching try block.

Line 118: `return 3`

Explanation: Returns the result from `max_parallel_jobs`.

Line 119: [blank]

Explanation: Blank line used to separate logical sections.

Line 120: [blank]

Explanation: Blank line used to separate logical sections.

Line 121: `def app_credentials() -> tuple[str, str]:`

Explanation: Reads the app username and password from `.env`.

Line 122: `env = read_env()`

Explanation: Assigns or updates the value of `env`.

Line 123: `return env.get("APP_USERNAME", "").strip(), env.get("APP_PASSWORD", "").strip()`

Explanation: Returns the result from `app_credentials`.

Line 124: [blank]

Explanation: Blank line used to separate logical sections.

Line 125: [blank]

Explanation: Blank line used to separate logical sections.

Line 126: `def auth_enabled() -> bool:`

Explanation: Decides whether login protection should be active.

Line 127: `username, password = app_credentials()`

Explanation: Continues the logic inside `auth_enabled`.

Line 128: `return bool(username and password and password != "change-this-password")`

Explanation: Returns the result from `auth_enabled`.

Line 129: [blank]

Explanation: Blank line used to separate logical sections.

Line 130: [blank]

Explanation: Blank line used to separate logical sections.

Line 131: `def authorized() -> bool:`

Explanation: Checks the incoming Basic Auth username and password.

Line 132: `if not auth_enabled():`

Explanation: Starts a conditional branch; the indented block runs only when this condition is true.

Line 133: `return True`

Explanation: Returns the result from `authorized`.

Line 134: `username, password = app_credentials()`

Explanation: Continues the logic inside `authorized`.

Line 135: `auth = request.authorization`

Explanation: Assigns or updates the value of `auth`.

Line 136: `if not auth:`

Explanation: Starts a conditional branch; the indented block runs only when this condition is true.

Line 137: `return False`

Explanation: Returns the result from `authorized`.

Line 138: return secrets.compare_digest(auth.username or "", username) and secrets.compare_digest(auth.password or "", password)

Explanation: Returns the result from authorized.

Line 139: [blank]

Explanation: Blank line used to separate logical sections.

Line 140: [blank]

Explanation: Blank line used to separate logical sections.

Line 141: @app.before_request

Explanation: Registers the following function as a Flask route or request hook.

Line 142: def require_auth() -> Response | None:

Explanation: Blocks protected routes unless the user is logged in.

Line 143: if request.endpoint == "health":

Explanation: Starts a conditional branch; the indented block runs only when this condition is true.

Line 144: return None

Explanation: Returns the result from require_auth.

Line 145: if authorized():

Explanation: Starts a conditional branch; the indented block runs only when this condition is true.

Line 146: return None

Explanation: Returns the result from require_auth.

Line 147: return Response(

Explanation: Returns the result from require_auth.

Line 148: "Authentication required",

Explanation: Continues the logic inside require_auth.

Line 149: status=401,

Explanation: Assigns or updates the value of status.

Line 150: headers={"WWW-Authenticate": 'Basic realm="Ayurvedic Book Processor"'},

Explanation: Assigns or updates the value of headers.

Line 151:)

Explanation: Closes a multi-line expression or data structure.

Line 152: [blank]

Explanation: Blank line used to separate logical sections.

Line 153: [blank]

Explanation: Blank line used to separate logical sections.

Line 154: @app.after_request

Explanation: Registers the following function as a Flask route or request hook.

Line 155: def add_production_headers(response: Response) -> Response:

Explanation: Adds browser security headers to every response.

Line 156: response.headers.setdefault("X-Content-Type-Options", "nosniff")

Explanation: Continues the logic inside add_production_headers.

Line 157: response.headers.setdefault("X-Frame-Options", "DENY")

Explanation: Continues the logic inside add_production_headers.

Line 158: response.headers.setdefault("Referrer-Policy", "no-referrer")

Explanation: Continues the logic inside add_production_headers.

Line 159: response.headers.setdefault("Cache-Control", "no-store")

Explanation: Continues the logic inside add_production_headers.

Line 160: return response

Explanation: Returns the result from add_production_headers.

Line 161: [blank]

Explanation: Blank line used to separate logical sections.

Line 162: [blank]

Explanation: Blank line used to separate logical sections.

Line 163: `@app.errorhandler(RequestEntityTooLarge)`

Explanation: Registers the following function as a Flask route or request hook.

Line 164: `def upload_too_large(_: RequestEntityTooLarge) -> Response:`

Explanation: Shows a friendly error when an uploaded file exceeds the size limit.

Line 165: `max_mb = int(app.config.get("MAX_CONTENT_LENGTH", 250 * 1024 * 1024) / (1024 * 1024))`

Explanation: Assigns or updates the value of max_mb.

Line 166: `return redirect(url_for("index", error=f"Upload is too large. Maximum allowed size is {max_mb} MB."))`

Explanation: Returns the result from upload_too_large.

Line 167: [blank]

Explanation: Blank line used to separate logical sections.

Line 168: [blank]

Explanation: Blank line used to separate logical sections.

Line 169: `@app.before_request`

Explanation: Registers the following function as a Flask route or request hook.

Line 170: `def start_runtime() -> None:`

Explanation: Ensures project folders and the background dispatcher exist before requests run.

Line 171: `if request.endpoint != "health":`

Explanation: Starts a conditional branch; the indented block runs only when this condition is true.

Line 172: `ensure_project_files()`

Explanation: Continues the logic inside start_runtime.

Line 173: `ensure_dispatcher()`

Explanation: Continues the logic inside start_runtime.

Line 174: [blank]

Explanation: Blank line used to separate logical sections.

Line 175: [blank]

Explanation: Blank line used to separate logical sections.

Line 176: `def parse_int_field(value: str | None, default: int, minimum: int, maximum: int) -> str:`

Explanation: Parses numeric form inputs and clamps them to a safe range.

Line 177: `try:`

Explanation: Starts a protected block where errors can be caught.

Line 178: `parsed = int(value or default)`

Explanation: Assigns or updates the value of parsed.

Line 179: `except ValueError:`

Explanation: Handles an error raised inside the matching try block.

Line 180: `parsed = default`

Explanation: Assigns or updates the value of parsed.

Line 181: `return str(max(minimum, min(maximum, parsed)))`

Explanation: Returns the result from parse_int_field.

Line 182: [blank]

Explanation: Blank line used to separate logical sections.

Line 183: [blank]

Explanation: Blank line used to separate logical sections.

Line 184: `def safe_pdf_name(filename: str) -> str:`

Explanation: Sanitizes uploaded PDF filenames for Windows-safe storage.

Line 185: `name = Path(filename).name.strip()`

Explanation: Assigns or updates the value of name.

Line 186: `for char in '<>:"/\\|?*':`

Explanation: Starts a loop over each item in a collection.

Line 187: name = name.replace(char, "_")
Explanation: Assigns or updates the value of name.

Line 188: if not name.lower().endswith(".pdf"):
Explanation: Starts a conditional branch; the indented block runs only when this condition is true.

Line 189: name += ".pdf"
Explanation: Continues the logic inside safe_pdf_name.

Line 190: return name or f"uploaded_{datetime.now().strftime('%Y%m%d_%H%M%S')}.pdf"
Explanation: Returns the result from safe_pdf_name.

Line 191: [blank]
Explanation: Blank line used to separate logical sections.

Line 192: [blank]
Explanation: Blank line used to separate logical sections.

Line 193: def looks_like_pdf(pdf_file: Any) -> bool:
Explanation: Checks the first bytes of an upload to confirm it looks like a real PDF.

Line 194: position = pdf_file.stream.tell()
Explanation: Assigns or updates the value of position.

Line 195: header = pdf_file.stream.read(5)
Explanation: Assigns or updates the value of header.

Line 196: pdf_file.stream.seek(position)
Explanation: Continues the logic inside looks_like_pdf.

Line 197: return header == b"%PDF-"
Explanation: Returns the result from looks_like_pdf.

Line 198: [blank]
Explanation: Blank line used to separate logical sections.

Line 199: [blank]
Explanation: Blank line used to separate logical sections.

Line 200: def load_jobs() -> list[dict[str, Any]]:
Explanation: Loads the job list from jobs/jobs.json.

Line 201: ensure_project_files()
Explanation: Continues the logic inside load_jobs.

Line 202: try:
Explanation: Starts a protected block where errors can be caught.

Line 203: data = json.loads(JOBS_DB.read_text(encoding="utf-8"))
Explanation: Reads text content from a file.

Line 204: return data if isinstance(data, list) else []
Explanation: Returns the result from load_jobs.

Line 205: except Exception:
Explanation: Handles an error raised inside the matching try block.

Line 206: return []
Explanation: Returns the result from load_jobs.

Line 207: [blank]
Explanation: Blank line used to separate logical sections.

Line 208: [blank]
Explanation: Blank line used to separate logical sections.

Line 209: def save_jobs(jobs: list[dict[str, Any]]) -> None:
Explanation: Writes the job list safely using a temporary file and replace.

Line 210: JOBS_DIR.mkdir(parents=True, exist_ok=True)
Explanation: Creates a folder if it does not already exist.

Line 211: temp = JOBS_DB.with_suffix(".json.tmp")
Explanation: Assigns or updates the value of temp.

Line 212: `temp.write_text(json.dumps(jobs, ensure_ascii=False, indent=2), encoding="utf-8")`

Explanation: Writes text content to a file.

Line 213: `temp.replace(JOBS_DB)`

Explanation: Continues the logic inside `save_jobs`.

Line 214: [blank]

Explanation: Blank line used to separate logical sections.

Line 215: [blank]

Explanation: Blank line used to separate logical sections.

Line 216: `def update_job(job_id: str, **updates: Any) -> None:`

Explanation: Updates one job record inside `jobs/jobs.json`.

Line 217: `with jobs_lock:`

Explanation: Opens a context-managed resource and closes it automatically afterward.

Line 218: `jobs = load_jobs()`

Explanation: Assigns or updates the value of `jobs`.

Line 219: `for job in jobs:`

Explanation: Starts a loop over each item in a collection.

Line 220: `if job["id"] == job_id:`

Explanation: Starts a conditional branch; the indented block runs only when this condition is true.

Line 221: `job.update(updates)`

Explanation: Continues the logic inside `update_job`.

Line 222: `job["updated_at"] = now_text()`

Explanation: Assigns or updates the value of `job["updated_at"]`.

Line 223: `break`

Explanation: Continues the logic inside `update_job`.

Line 224: `save_jobs(jobs)`

Explanation: Continues the logic inside `update_job`.

Line 225: [blank]

Explanation: Blank line used to separate logical sections.

Line 226: [blank]

Explanation: Blank line used to separate logical sections.

Line 227: `def job_dir(job_id: str) -> Path:`

Explanation: Returns the folder path for a job id.

Line 228: `return JOBS_DIR / job_id`

Explanation: Returns the result from `job_dir`.

Line 229: [blank]

Explanation: Blank line used to separate logical sections.

Line 230: [blank]

Explanation: Blank line used to separate logical sections.

Line 231: `def create_job(pdf_file: Any) -> None:`

Explanation: Creates a new job folder, saves the uploaded PDF, and queues the job.

Line 232: `env = read_env()`

Explanation: Assigns or updates the value of `env`.

Line 233: `job_id = datetime.now().strftime("%Y%m%d_%H%M%S_") + uuid.uuid4().hex[:8]`

Explanation: Assigns or updates the value of `job_id`.

Line 234: `folder = job_dir(job_id)`

Explanation: Assigns or updates the value of `folder`.

Line 235: `pdf_dir = folder / "pdfs"`

Explanation: Assigns or updates the value of `pdf_dir`.

Line 236: `pdf_dir.mkdir(parents=True, exist_ok=True)`

Explanation: Creates a folder if it does not already exist.

Line 237: filename = safe_pdf_name(pdf_file.filename or "uploaded.pdf")

Explanation: Assigns or updates the value of filename.

Line 238: pdf_file.save(pdf_dir / filename)

Explanation: Continues the logic inside create_job.

Line 239: [blank]

Explanation: Blank line used to separate logical sections.

Line 240: job_env = env.copy()

Explanation: Assigns or updates the value of job_env.

Line 241: job_env.update(

Explanation: Continues the logic inside create_job.

Line 242: {

Explanation: Starts or continues a multi-line expression or data structure.

Line 243: "PDF_DIR": "./pdfs",

Explanation: Continues the logic inside create_job.

Line 244: "OUTPUT_DIR": "./output_notes",

Explanation: Continues the logic inside create_job.

Line 245: "CREATE_DOCX": "true",

Explanation: Continues the logic inside create_job.

Line 246: "RESET_MASTER": "false",

Explanation: Continues the logic inside create_job.

Line 247: "TEST_START_PAGE": env.get("TEST_START_PAGE", "1"),

Explanation: Continues the logic inside create_job.

Line 248: "TEST_MAX_PAGES": env.get("TEST_MAX_PAGES", "0"),

Explanation: Continues the logic inside create_job.

Line 249: "TEST_PDF_LIMIT": "1",

Explanation: Continues the logic inside create_job.

Line 250: "CREATE_STRUCTURED_NOTES": env.get("CREATE_STRUCTURED_NOTES", "false"),

Explanation: Continues the logic inside create_job.

Line 251: "SPEED_MODE": env.get("SPEED_MODE", "balanced"),

Explanation: Continues the logic inside create_job.

Line 252: "PAGE_WORKERS_PER_JOB": env.get("PAGE_WORKERS_PER_JOB", "2"),

Explanation: Continues the logic inside create_job.

Line 253: }

Explanation: Closes a multi-line expression or data structure.

Line 254:)

Explanation: Closes a multi-line expression or data structure.

Line 255: env_lines = [f"{key}={value}" for key, value in job_env.items()]

Explanation: Assigns or updates the value of env_lines.

Line 256: (folder / ".env").write_text("\n".join(env_lines) + "\n", encoding="utf-8")

Explanation: Writes text content to a file.

Line 257: [blank]

Explanation: Blank line used to separate logical sections.

Line 258: job = {

Explanation: Assigns or updates the value of job.

Line 259: "id": job_id,

Explanation: Continues the logic inside create_job.

Line 260: "filename": filename,

Explanation: Continues the logic inside create_job.

Line 261: "status": "queued",

Explanation: Continues the logic inside create_job.

Line 262: "created_at": now_text(),
Explanation: Continues the logic inside create_job.

Line 263: "updated_at": now_text(),
Explanation: Continues the logic inside create_job.

Line 264: "return_code": None,
Explanation: Continues the logic inside create_job.

Line 265: }
Explanation: Closes a multi-line expression or data structure.

Line 266: with jobs_lock:
Explanation: Opens a context-managed resource and closes it automatically afterward.

Line 267: jobs = load_jobs()
Explanation: Assigns or updates the value of jobs.

Line 268: jobs.insert(0, job)
Explanation: Continues the logic inside create_job.

Line 269: save_jobs(jobs)
Explanation: Continues the logic inside create_job.

Line 270: [blank]
Explanation: Blank line used to separate logical sections.

Line 271: [blank]
Explanation: Blank line used to separate logical sections.

Line 272: def job_output_files(job_id: str) -> list[Path]:
Explanation: Finds DOCX files produced by a job.

Line 273: output_dir = job_dir(job_id) / "output_notes"
Explanation: Assigns or updates the value of output_dir.

Line 274: if not output_dir.exists():
Explanation: Starts a conditional branch; the indented block runs only when this condition is true.

Line 275: return []
Explanation: Returns the result from job_output_files.

Line 276: return sorted(output_dir.glob("*.docx"), key=lambda path: path.stat().st_mtime, reverse=True)
Explanation: Returns the result from job_output_files.

Line 277: [blank]
Explanation: Blank line used to separate logical sections.

Line 278: [blank]
Explanation: Blank line used to separate logical sections.

Line 279: def completed_doc_files() -> list[Path]:
Explanation: Lists final DOCX files copied to completed_docs.

Line 280: ensure_project_files()
Explanation: Continues the logic inside completed_doc_files.

Line 281: return sorted(COMPLETED_DOCS_DIR.glob("*.docx"), key=lambda path: path.stat().st_mtime, reverse=True)
Explanation: Returns the result from completed_doc_files.

Line 282: [blank]
Explanation: Blank line used to separate logical sections.

Line 283: [blank]
Explanation: Blank line used to separate logical sections.

Line 284: def publish_docx_outputs(job_id: str) -> list[str]:
Explanation: Copies DOCX files from a job into completed_docs.

Line 285: published: list[str] = []
Explanation: Assigns or updates the value of published: list[str].

Line 286: for source in job_output_files(job_id):
Explanation: Starts a loop over each item in a collection.

Line 287: target_name = f"{job_id}_{source.name}"

Explanation: Assigns or updates the value of target_name.

Line 288: target = COMPLETED_DOCS_DIR / target_name

Explanation: Assigns or updates the value of target.

Line 289: shutil.copy2(source, target)

Explanation: Continues the logic inside publish_docx_outputs.

Line 290: published.append(target_name)

Explanation: Continues the logic inside publish_docx_outputs.

Line 291: return published

Explanation: Returns the result from publish_docx_outputs.

Line 292: [blank]

Explanation: Blank line used to separate logical sections.

Line 293: [blank]

Explanation: Blank line used to separate logical sections.

Line 294: def job_log(job_id: str, max_chars: int = 5000) -> str:

Explanation: Reads the recent log text for one job.

Line 295: log_path = job_dir(job_id) / "logs" / "interface_run.log"

Explanation: Assigns or updates the value of log_path.

Line 296: if not log_path.exists():

Explanation: Starts a conditional branch; the indented block runs only when this condition is true.

Line 297: log_path = job_dir(job_id) / "logs" / "processor.log"

Explanation: Assigns or updates the value of log_path.

Line 298: if not log_path.exists():

Explanation: Starts a conditional branch; the indented block runs only when this condition is true.

Line 299: return ""

Explanation: Returns the result from job_log.

Line 300: return log_path.read_text(encoding="utf-8", errors="replace")[-max_chars:]

Explanation: Returns the result from job_log.

Line 301: [blank]

Explanation: Blank line used to separate logical sections.

Line 302: [blank]

Explanation: Blank line used to separate logical sections.

Line 303: def summarize_failure(job_id: str) -> str:

Explanation: Extracts the most useful failure line from a job log.

Line 304: text = job_log(job_id, max_chars=12000)

Explanation: Assigns or updates the value of text.

Line 305: for line in reversed(text.splitlines()):

Explanation: Starts a loop over each item in a collection.

Line 306: stripped = line.strip()

Explanation: Assigns or updates the value of stripped.

Line 307: if "[ERROR]" in stripped:

Explanation: Starts a conditional branch; the indented block runs only when this condition is true.

Line 308: return stripped.split("[ERROR]", 1)[-1].strip()

Explanation: Returns the result from summarize_failure.

Line 309: if "Missing GEMINI_API_KEY" in stripped or "Could not read PDF pages" in stripped:

Explanation: Starts a conditional branch; the indented block runs only when this condition is true.

Line 310: return stripped

Explanation: Returns the result from summarize_failure.

Line 311: return "Processing finished but no Word document was produced. Check the recent log."

Explanation: Returns the result from summarize_failure.

Line 312: [blank]

Explanation: Blank line used to separate logical sections.

Line 313: [blank]

Explanation: Blank line used to separate logical sections.

Line 314: `def run_job(job: dict[str, Any]) -> None:`

Explanation: Starts ultimate_book_processor.py for one job and records completion state.

Line 315: `job_id = job["id"]`

Explanation: Assigns or updates the value of job_id.

Line 316: `folder = job_dir(job_id)`

Explanation: Assigns or updates the value of folder.

Line 317: `log_dir = folder / "logs"`

Explanation: Assigns or updates the value of log_dir.

Line 318: `log_dir.mkdir(parents=True, exist_ok=True)`

Explanation: Creates a folder if it does not already exist.

Line 319: `log_path = log_dir / "interface_run.log"`

Explanation: Assigns or updates the value of log_path.

Line 320: `env = os.environ.copy()`

Explanation: Assigns or updates the value of env.

Line 321: `env["PYTHONIOENCODING"] = "utf-8"`

Explanation: Assigns or updates the value of env["PYTHONIOENCODING"].

Line 322: [blank]

Explanation: Blank line used to separate logical sections.

Line 323: `update_job(job_id, status="running", started_at=now_text())`

Explanation: Continues the logic inside run_job.

Line 324: `with log_path.open("a", encoding="utf-8") as log:`

Explanation: Opens a context-managed resource and closes it automatically afterward.

Line 325: `log.write(f"=== Job started {now_text()} ===\n")`

Explanation: Assigns or updates the value of log.write(f".

Line 326: `log.flush()`

Explanation: Continues the logic inside run_job.

Line 327: `proc = subprocess.Popen(`

Explanation: Starts the processor as a separate background process for the job.

Line 328: `[sys.executable, str(PROCESSOR)],`

Explanation: Starts or continues a multi-line expression or data structure.

Line 329: `cwd=str(folder),`

Explanation: Assigns or updates the value of cwd.

Line 330: `stdout=log,`

Explanation: Assigns or updates the value of stdout.

Line 331: `stderr=subprocess.STDOUT,`

Explanation: Assigns or updates the value of stderr.

Line 332: `text=True,`

Explanation: Assigns or updates the value of text.

Line 333: `env=env,`

Explanation: Assigns or updates the value of env.

Line 334: `)`

Explanation: Closes a multi-line expression or data structure.

Line 335: `with jobs_lock:`

Explanation: Opens a context-managed resource and closes it automatically afterward.

Line 336: `running_processes[job_id] = proc`

Explanation: Assigns or updates the value of running_processes[job_id].

Line 337: return_code = proc.wait()

Explanation: Assigns or updates the value of return_code.

Line 338: log.write(f"\n=== Job finished with code {return_code} at {now_text()} ===\n")

Explanation: Assigns or updates the value of log.write(f"\n").

Line 339: [blank]

Explanation: Blank line used to separate logical sections.

Line 340: with jobs_lock:

Explanation: Opens a context-managed resource and closes it automatically afterward.

Line 341: running_processes.pop(job_id, None)

Explanation: Continues the logic inside run_job.

Line 342: outputs = job_output_files(job_id)

Explanation: Assigns or updates the value of outputs.

Line 343: published_outputs = publish_docx_outputs(job_id) if return_code == 0 else []

Explanation: Assigns or updates the value of published_outputs.

Line 344: update_job(

Explanation: Continues the logic inside run_job.

Line 345: job_id,

Explanation: Continues the logic inside run_job.

Line 346: status="completed" if return_code == 0 and outputs else "failed",

Explanation: Assigns or updates the value of status.

Line 347: return_code=return_code,

Explanation: Assigns or updates the value of return_code.

Line 348: finished_at=now_text(),

Explanation: Assigns or updates the value of finished_at.

Line 349: output_count=len(outputs),

Explanation: Assigns or updates the value of output_count.

Line 350: published_outputs=published_outputs,

Explanation: Assigns or updates the value of published_outputs.

Line 351: failure_reason="" if return_code == 0 and outputs else summarize_failure(job_id),

Explanation: Assigns or updates the value of failure_reason.

Line 352:)

Explanation: Closes a multi-line expression or data structure.

Line 353: [blank]

Explanation: Blank line used to separate logical sections.

Line 354: [blank]

Explanation: Blank line used to separate logical sections.

Line 355: def dispatcher_loop() -> None:

Explanation: Continuously starts queued jobs while respecting the parallel job limit.

Line 356: while True:

Explanation: Starts a loop that continues while the condition remains true.

Line 357: with jobs_lock:

Explanation: Opens a context-managed resource and closes it automatically afterward.

Line 358: jobs = load_jobs()

Explanation: Assigns or updates the value of jobs.

Line 359: running_ids = [job_id for job_id, proc in running_processes.items() if proc.poll() is None]

Explanation: Assigns or updates the value of running_ids.

Line 360: for job_id in list(running_processes):

Explanation: Starts a loop over each item in a collection.

Line 361: if job_id not in running_ids:

Explanation: Starts a conditional branch; the indented block runs only when this condition is true.

Line 362: `running_processes.pop(job_id, None)`
Explanation: Continues the logic inside dispatcher_loop.

Line 363: `slots = max_parallel_jobs() - len(running_processes)`
Explanation: Assigns or updates the value of slots.

Line 364: `queued = sorted(`
Explanation: Assigns or updates the value of queued.

Line 365: `[job for job in jobs if job.get("status") == "queued"],`
Explanation: Assigns or updates the value of [job for job in jobs if job.get("status").

Line 366: `key=lambda job: job.get("created_at", ""),`
Explanation: Assigns or updates the value of key.

Line 367: `)`
Explanation: Closes a multi-line expression or data structure.

Line 368: [blank]
Explanation: Blank line used to separate logical sections.

Line 369: `for job in queued[: max(0, slots)]:`
Explanation: Starts a loop over each item in a collection.

Line 370: `threading.Thread(target=run_job, args=(job,), daemon=True).start()`
Explanation: Starts work on a background thread.

Line 371: [blank]
Explanation: Blank line used to separate logical sections.

Line 372: `threading.Event().wait(3)`
Explanation: Continues the logic inside dispatcher_loop.

Line 373: [blank]
Explanation: Blank line used to separate logical sections.

Line 374: [blank]
Explanation: Blank line used to separate logical sections.

Line 375: `def ensure_dispatcher() -> None:`
Explanation: Starts the dispatcher once and marks stale running jobs after restart.

Line 376: `global dispatcher_started`
Explanation: Continues the logic inside ensure_dispatcher.

Line 377: `with dispatcher_lock:`
Explanation: Opens a context-managed resource and closes it automatically afterward.

Line 378: `if dispatcher_started:`
Explanation: Starts a conditional branch; the indented block runs only when this condition is true.

Line 379: `return`
Explanation: Continues the logic inside ensure_dispatcher.

Line 380: `with jobs_lock:`
Explanation: Opens a context-managed resource and closes it automatically afterward.

Line 381: `jobs = load_jobs()`
Explanation: Assigns or updates the value of jobs.

Line 382: `changed = False`
Explanation: Assigns or updates the value of changed.

Line 383: `for job in jobs:`
Explanation: Starts a loop over each item in a collection.

Line 384: `if job.get("status") == "running":`
Explanation: Starts a conditional branch; the indented block runs only when this condition is true.

Line 385: `job["status"] = "failed"`
Explanation: Assigns or updates the value of job["status"].

Line 386: `job["return_code"] = None`
Explanation: Assigns or updates the value of job["return_code"].

Line 387: job["failure_reason"] = "Server restarted while this job was marked running. Use Run Again to resume it."
Explanation: Assigns or updates the value of job["failure_reason"].

Line 388: job["updated_at"] = now_text()
Explanation: Assigns or updates the value of job["updated_at"].

Line 389: changed = True
Explanation: Assigns or updates the value of changed.

Line 390: if changed:
Explanation: Starts a conditional branch; the indented block runs only when this condition is true.

Line 391: save_jobs(jobs)
Explanation: Continues the logic inside ensure_dispatcher.

Line 392: dispatcher_started = True
Explanation: Assigns or updates the value of dispatcher_started.

Line 393: threading.Thread(target=dispatcher_loop, daemon=True).start()
Explanation: Starts work on a background thread.

Line 394: [blank]
Explanation: Blank line used to separate logical sections.

Line 395: [blank]
Explanation: Blank line used to separate logical sections.

Line 396: def reset_job(job_id: str) -> bool:
Explanation: Moves a failed or completed job back to queued so it can resume.

Line 397: if not valid_job_id(job_id):
Explanation: Starts a conditional branch; the indented block runs only when this condition is true.

Line 398: return False
Explanation: Returns the result from reset_job.

Line 399: jobs = load_jobs()
Explanation: Assigns or updates the value of jobs.

Line 400: found = False
Explanation: Assigns or updates the value of found.

Line 401: for job in jobs:
Explanation: Starts a loop over each item in a collection.

Line 402: if job["id"] == job_id and job.get("status") in {"failed", "completed"}:
Explanation: Starts a conditional branch; the indented block runs only when this condition is true.

Line 403: job["status"] = "queued"
Explanation: Assigns or updates the value of job["status"].

Line 404: job["return_code"] = None
Explanation: Assigns or updates the value of job["return_code"].

Line 405: job["updated_at"] = now_text()
Explanation: Assigns or updates the value of job["updated_at"].

Line 406: found = True
Explanation: Assigns or updates the value of found.

Line 407: save_jobs(jobs)
Explanation: Continues the logic inside reset_job.

Line 408: return found
Explanation: Returns the result from reset_job.

Line 409: [blank]
Explanation: Blank line used to separate logical sections.

Line 410: [blank]
Explanation: Blank line used to separate logical sections.

Line 411: `def delete_job(job_id: str) -> None:`

Explanation: Stops and removes a non-running job and its folder.

Line 412: `if not valid_job_id(job_id):`

Explanation: Starts a conditional branch; the indented block runs only when this condition is true.

Line 413: `return`

Explanation: Continues the logic inside `delete_job`.

Line 414: `with jobs_lock:`

Explanation: Opens a context-managed resource and closes it automatically afterward.

Line 415: `proc = running_processes.get(job_id)`

Explanation: Assigns or updates the value of `proc`.

Line 416: `if proc and proc.poll() is None:`

Explanation: Starts a conditional branch; the indented block runs only when this condition is true.

Line 417: `proc.terminate()`

Explanation: Continues the logic inside `delete_job`.

Line 418: `jobs = [job for job in load_jobs() if job["id"] != job_id]`

Explanation: Assigns or updates the value of `jobs`.

Line 419: `save_jobs(jobs)`

Explanation: Continues the logic inside `delete_job`.

Line 420: `folder = job_dir(job_id)`

Explanation: Assigns or updates the value of `folder`.

Line 421: `if folder.exists() and folder.resolve().is_relative_to(JOBS_DIR.resolve()):`

Explanation: Starts a conditional branch; the indented block runs only when this condition is true.

Line 422: `shutil.rmtree(folder)`

Explanation: Continues the logic inside `delete_job`.

Line 423: [blank]

Explanation: Blank line used to separate logical sections.

Line 424: [blank]

Explanation: Blank line used to separate logical sections.

Line 425: `def render_page() -> str:`

Explanation: Builds the complete HTML page shown in the browser.

Line 426: `ensure_dispatcher()`

Explanation: Continues the logic inside `render_page`.

Line 427: `env = read_env()`

Explanation: Assigns or updates the value of `env`.

Line 428: `key_ready = bool(env.get("GEMINI_API_KEY")) and env.get("GEMINI_API_KEY") != "your_key_here"`

Explanation: Assigns or updates the value of `key_ready`.

Line 429: `structured_text = "On" if env.get("CREATE_STRUCTURED_NOTES", "false").lower() == "true" else "Off"`

Explanation: Assigns or updates the value of `structured_text`.

Line 430: `speed_mode = env.get("SPEED_MODE", "balanced").lower()`

Explanation: Assigns or updates the value of `speed_mode`.

Line 431: `page_workers = env.get("PAGE_WORKERS_PER_JOB", "2")`

Explanation: Assigns or updates the value of `page_workers`.

Line 432: `test_start_page = env.get("TEST_START_PAGE", "1")`

Explanation: Assigns or updates the value of `test_start_page`.

Line 433: `test_max_pages = env.get("TEST_MAX_PAGES", "0")`

Explanation: Assigns or updates the value of `test_max_pages`.

Line 434: `jobs = load_jobs()`

Explanation: Assigns or updates the value of `jobs`.

Line 435: `running_count = sum(1 for job in jobs if job.get("status") == "running")`

Explanation: Assigns or updates the value of `running_count`.

Line 436: queued_count = sum(1 for job in jobs if job.get("status") == "queued")

Explanation: Assigns or updates the value of queued_count.

Line 437: completed_docs = completed_doc_files()

Explanation: Assigns or updates the value of completed_docs.

Line 438: upload_message = request.args.get("uploaded")

Explanation: Assigns or updates the value of upload_message.

Line 439: error_message = request.args.get("error")

Explanation: Assigns or updates the value of error_message.

Line 440: [blank]

Explanation: Blank line used to separate logical sections.

Line 441: job_items = []

Explanation: Assigns or updates the value of job_items.

Line 442: for job in jobs[:80]:

Explanation: Starts a loop over each item in a collection.

Line 443: outputs = [COMPLETED_DOCS_DIR / name for name in job.get("published_outputs", [])]

Explanation: Assigns or updates the value of outputs.

Line 444: if not outputs:

Explanation: Starts a conditional branch; the indented block runs only when this condition is true.

Line 445: outputs = job_output_files(job["id"])

Explanation: Assigns or updates the value of outputs.

Line 446: links = "".join(

Explanation: Assigns or updates the value of links.

Line 447: f"<a class='button secondary' href='{url_for('download_output', job_id=job['id'],
filename=path.name)}'>Download DOCX"

Explanation: Builds a URL for a named Flask route.

Line 448: for path in outputs

Explanation: Starts a loop over each item in a collection.

Line 449:)

Explanation: Closes a multi-line expression or data structure.

Line 450: actions = links

Explanation: Assigns or updates the value of actions.

Line 451: if job.get("status") in {"failed", "completed"}:

Explanation: Starts a conditional branch; the indented block runs only when this condition is true.

Line 452: actions += f"<form action='{url_for('retry_job', job_id=job['id'])}' method='post'><button
class='secondary'>Run Again / Resume</button></form>"

Explanation: Builds a URL for a named Flask route.

Line 453: if job.get("status") != "running":

Explanation: Starts a conditional branch; the indented block runs only when this condition is true.

Line 454: actions += f"<form action='{url_for('remove_job', job_id=job['id'])}' method='post'><button
class='danger'>Delete</button></form>"

Explanation: Builds a URL for a named Flask route.

Line 455: status_class = f"status {job.get('status', 'queued')}"

Explanation: Assigns or updates the value of status_class.

Line 456: failure = f"<small class='problem'>{escape(job.get('failure_reason', ''))}</small>" if
job.get("failure_reason") else ""

Explanation: Assigns or updates the value of failure.

Line 457: job_items.append(

Explanation: Continues the logic inside render_page.

Line 458: f"<div>{escape(job['filename'])}<small><span
class='{status_class}'>{escape(job['status'])} created {escape(job['created_at'])} | id
{escape(job['id'])}</small><small>{failure}</div>"

Explanation: Assigns or updates the value of f"<div>{escape(job['filename'])}<small><span class.

Line 459: `f"<div class='actions inline'>{actions}</div></li>"`

Explanation: Continues the logic inside `render_page`.

Line 460: `)`

Explanation: Closes a multi-line expression or data structure.

Line 461: `jobs_html = "".join(job_items) or "<li><div><strong>No jobs yet</strong><small>Upload PDFs to start conversion automatically.</small></div></li>"`

Explanation: Assigns or updates the value of `jobs_html`.

Line 462: `docs_html = "".join(`

Explanation: Assigns or updates the value of `docs_html`.

Line 463: `f"<li><div><strong>{escape(path.name)}</strong><small>{round(path.stat().st_size / (1024 * 1024), 2)} MB | {datetime.fromtimestamp(path.stat().st_mtime).strftime('%Y-%m-%d %H:%M')}</small></div></li>"`

Explanation: Continues the logic inside `render_page`.

Line 464: `f"<div class='actions inline'><a class='button secondary' href='{url_for('download_completed_doc', filename=path.name)}'>Download Word File</a></div></li>"`

Explanation: Builds a URL for a named Flask route.

Line 465: `for path in completed_docs[:60]`

Explanation: Starts a loop over each item in a collection.

Line 466: `) or "<li><div><strong>No Word files yet</strong><small>Completed jobs will appear here.</small></div></li>"`

Explanation: Continues the logic inside `render_page`.

Line 467: `notice_html = ""`

Explanation: Assigns or updates the value of `notice_html`.

Line 468: `if upload_message:`

Explanation: Starts a conditional branch; the indented block runs only when this condition is true.

Line 469: `notice_html = f"<div class='notice success'>{escape(upload_message)} PDF file(s) uploaded and queued for processing.</div>"`

Explanation: Assigns or updates the value of `notice_html`.

Line 470: `elif error_message:`

Explanation: Starts another conditional branch if the previous conditions were false.

Line 471: `notice_html = f"<div class='notice error'>{escape(error_message)}</div>"`

Explanation: Assigns or updates the value of `notice_html`.

Line 472: `[blank]`

Explanation: Blank line used to separate logical sections.

Line 473: `return f"""<!doctype html>`

Explanation: Returns the result from `render_page`.

Line 474: `<html lang="en">`

Explanation: Continues the logic inside `render_page`.

Line 475: `<head>`

Explanation: Continues the logic inside `render_page`.

Line 476: `<meta charset="utf-8">`

Explanation: Continues the logic inside `render_page`.

Line 477: `<meta name="viewport" content="width=device-width, initial-scale=1">`

Explanation: Continues the logic inside `render_page`.

Line 478: `<title>Book Processor</title>`

Explanation: Continues the logic inside `render_page`.

Line 479: `<style>`

Explanation: Continues the logic inside `render_page`.

Line 480: `:root {{`

Explanation: Continues the logic inside `render_page`.

Line 481: `--bg: #f5f6f8; --panel: #fff; --text: #1f2933; --muted: #697586;`

Explanation: Continues the logic inside `render_page`.

Line 482: --line: #d9dee5; --accent: #0f766e; --accent-dark: #115e59; --bad: #b42318; --soft: #e8f3f1; --blue: #175cd3;

Explanation: Continues the logic inside render_page.

Line 483: }}

Explanation: Continues the logic inside render_page.

Line 484: * {{ box-sizing: border-box; }}

Explanation: Continues the logic inside render_page.

Line 485: body {{ margin: 0; background: var(--bg); color: var(--text); font-family: Segoe UI, Arial, sans-serif; font-size: 15px; }}

Explanation: Continues the logic inside render_page.

Line 486: header {{ background: linear-gradient(180deg, #ffffff 0%, #f7fbfa 100%); border-bottom: 1px solid var(--line); padding: 18px 28px; display: flex; justify-content: space-between; gap: 18px; align-items: center; }}

Explanation: Continues the logic inside render_page.

Line 487: h1 {{ margin: 0; font-size: 22px; }}

Explanation: Continues the logic inside render_page.

Line 488: main {{ max-width: 1180px; margin: 0 auto; padding: 24px; display: grid; gap: 18px; }}

Explanation: Continues the logic inside render_page.

Line 489: .grid {{ display: grid; grid-template-columns: repeat(2, minmax(0, 1fr)); gap: 18px; }}

Explanation: Continues the logic inside render_page.

Line 490: section {{ background: var(--panel); border: 1px solid var(--line); border-radius: 8px; padding: 18px; box-shadow: 0 1px 2px rgba(16, 24, 40, .04); }}

Explanation: Continues the logic inside render_page.

Line 491: h2 {{ margin: 0 0 14px; font-size: 17px; }}

Explanation: Continues the logic inside render_page.

Line 492: label {{ display: block; margin: 12px 0 6px; color: var(--muted); font-size: 13px; }}

Explanation: Continues the logic inside render_page.

Line 493: input[type=file], input[type=number], select {{ width: 100%; border: 1px solid var(--line); border-radius: 6px; padding: 10px; background: #fff; }}

Explanation: Assigns or updates the value of input[type].

Line 494: .button, button {{ border: 0; border-radius: 6px; background: var(--accent); color: #fff; padding: 10px 14px; font-weight: 650; cursor: pointer; text-decoration: none; display: inline-flex; align-items: center; justify-content: center; min-height: 40px; }}

Explanation: Continues the logic inside render_page.

Line 495: .button:hover, button:hover {{ background: var(--accent-dark); }}

Explanation: Continues the logic inside render_page.

Line 496: .secondary {{ background: #eef2f6; color: var(--text); border: 1px solid var(--line); }}

Explanation: Continues the logic inside render_page.

Line 497: .secondary:hover {{ background: #e4e9f0; }}

Explanation: Continues the logic inside render_page.

Line 498: .danger {{ background: var(--bad); }}

Explanation: Continues the logic inside render_page.

Line 499: .danger:hover {{ background: #912018; }}

Explanation: Continues the logic inside render_page.

Line 500: .actions {{ display: flex; flex-wrap: wrap; gap: 10px; margin-top: 14px; }}

Explanation: Continues the logic inside render_page.

Line 501: .actions.inline {{ margin-top: 0; justify-content: flex-end; }}

Explanation: Continues the logic inside render_page.

Line 502: .statusbar {{ display: flex; flex-wrap: wrap; gap: 10px; align-items: center; }}

Explanation: Continues the logic inside render_page.

Line 503: .pill {{ border: 1px solid var(--line); background: #fff; padding: 8px 10px; border-radius: 999px; color: var(--muted); font-size: 13px; }}

Explanation: Continues the logic inside render_page.

Line 504: `.pill.ready {{ background: var(--soft); border-color: #b7ddd6; }}`

Explanation: Continues the logic inside `render_page`.

Line 505: `.notice {{ border-radius: 8px; padding: 12px 14px; border: 1px solid var(--line); background: #fff; }}`

Explanation: Continues the logic inside `render_page`.

Line 506: `.notice.success {{ background: #ecfdf3; border-color: #abefc6; color: #067647; }}`

Explanation: Continues the logic inside `render_page`.

Line 507: `.notice.error {{ background: #fef3f2; border-color: #fecdc6; color: var(--bad); }}`

Explanation: Continues the logic inside `render_page`.

Line 508: `.progressbox {{ border: 1px solid #b7ddd6; background: #f0fdfa; border-radius: 8px; padding: 14px; display: flex; justify-content: space-between; gap: 12px; align-items: center; }}`

Explanation: Continues the logic inside `render_page`.

Line 509: `.progressbox.idle {{ border-color: var(--line); background: #fff; }}`

Explanation: Continues the logic inside `render_page`.

Line 510: `.spinner {{ width: 18px; height: 18px; border: 3px solid #b7ddd6; border-top-color: var(--accent); border-radius: 50%; animation: spin 1s linear infinite; flex: none; }}`

Explanation: Continues the logic inside `render_page`.

Line 511: `.status {{ display: inline-block; border-radius: 999px; padding: 3px 8px; margin-right: 6px; font-weight: 700; text-transform: uppercase; font-size: 11px; }}`

Explanation: Continues the logic inside `render_page`.

Line 512: `.status.queued {{ background: #eff8ff; color: #175cd3; }}`

Explanation: Continues the logic inside `render_page`.

Line 513: `.status.running {{ background: #f0fdfa; color: #0f766e; }}`

Explanation: Continues the logic inside `render_page`.

Line 514: `.status.completed {{ background: #ecfdf3; color: #067647; }}`

Explanation: Continues the logic inside `render_page`.

Line 515: `.status.failed {{ background: #fef3f2; color: #b42318; }}`

Explanation: Continues the logic inside `render_page`.

Line 516: `@keyframes spin {{ to {{ transform: rotate(360deg); }} }}`

Explanation: Continues the logic inside `render_page`.

Line 517: `.steps {{ display: grid; grid-template-columns: repeat(3, 1fr); gap: 10px; }}`

Explanation: Continues the logic inside `render_page`.

Line 518: `.step {{ border: 1px solid var(--line); background: #fff; border-radius: 8px; padding: 12px; }}`

Explanation: Continues the logic inside `render_page`.

Line 519: `.step b {{ display: block; margin-bottom: 3px; }}`

Explanation: Continues the logic inside `render_page`.

Line 520: `.pill strong {{ color: var(--text); }}`

Explanation: Continues the logic inside `render_page`.

Line 521: `.problem {{ color: var(--bad); }}`

Explanation: Continues the logic inside `render_page`.

Line 522: `ul {{ margin: 0; padding: 0; list-style: none; display: grid; gap: 8px; }}`

Explanation: Continues the logic inside `render_page`.

Line 523: `li {{ border: 1px solid var(--line); border-radius: 6px; padding: 10px; display: grid; grid-template-columns: 1fr auto; gap: 12px; align-items: center; min-height: 56px; }}`

Explanation: Continues the logic inside `render_page`.

Line 524: `small {{ display: block; color: var(--muted); margin-top: 3px; }}`

Explanation: Continues the logic inside `render_page`.

Line 525: `pre {{ margin: 0; max-height: 320px; overflow: auto; background: #101828; color: #e6edf3; padding: 14px; border-radius: 8px; white-space: pre-wrap; font-family: Consolas, monospace; font-size: 12px; }}`

Explanation: Continues the logic inside `render_page`.

Line 526: `.wide {{ grid-column: 1 / -1; }}`

Explanation: Continues the logic inside `render_page`.

Line 527: `.note {{ color: var(--muted); margin-top: 8px; }}`

Explanation: Continues the logic inside `render_page`.

Line 528: `@media (max-width: 760px) {{ header {{ flex-direction: column; align-items: flex-start; }} main {{ padding: 16px; }} .grid {{ grid-template-columns: 1fr; }} li {{ grid-template-columns: 1fr; }} .actions.inline {{ justify-content: flex-start; }} }}`

Explanation: Continues the logic inside `render_page`.

Line 529: `</style>`

Explanation: Continues the logic inside `render_page`.

Line 530: `</head>`

Explanation: Continues the logic inside `render_page`.

Line 531: `<body>`

Explanation: Continues the logic inside `render_page`.

Line 532: `<header>`

Explanation: Continues the logic inside `render_page`.

Line 533: `<div>`

Explanation: Continues the logic inside `render_page`.

Line 534: `<h1>Ayurvedic Book Processor</h1>`

Explanation: Continues the logic inside `render_page`.

Line 535: `<div class="note">Gemini multimodal transcription + verification | parallel DOCX jobs</div>`

Explanation: Continues the logic inside `render_page`.

Line 536: `</div>`

Explanation: Continues the logic inside `render_page`.

Line 537: `<div class="statusbar">`

Explanation: Continues the logic inside `render_page`.

Line 538: `<span class="pill ready">Mode: <strong>{'Full PDF' if test_max_pages == '0' else escape(test_max_pages) + ' pages'}</strong></span>`

Explanation: Continues the logic inside `render_page`.

Line 539: `<span class="pill">Structured notes: <strong>{structured_text}</strong></span>`

Explanation: Continues the logic inside `render_page`.

Line 540: `<span class="pill ready">Speed: <strong>{speed_mode.title()}</strong></span>`

Explanation: Continues the logic inside `render_page`.

Line 541: `<span class="pill ready">Parallel jobs: <strong>{max_parallel_jobs()}</strong></span>`

Explanation: Continues the logic inside `render_page`.

Line 542: `<span class="pill {'problem' if not key_ready else 'ready'}">API key: <strong>{'Set' if key_ready else 'Missing'}</strong></span>`

Explanation: Continues the logic inside `render_page`.

Line 543: `<span class="pill {'ready' if auth_enabled() else 'problem'}">Login: <strong>{'On' if auth_enabled() else 'Off'}</strong></span>`

Explanation: Continues the logic inside `render_page`.

Line 544: `</div>`

Explanation: Continues the logic inside `render_page`.

Line 545: `</header>`

Explanation: Continues the logic inside `render_page`.

Line 546: `<main>`

Explanation: Continues the logic inside `render_page`.

Line 547: `{notice_html}`

Explanation: Starts or continues a multi-line expression or data structure.

Line 548: `<section>`

Explanation: Continues the logic inside `render_page`.

Line 549: `<h2>Workflow</h2>`

Explanation: Continues the logic inside `render_page`.

Line 550: <div class="steps">

Explanation: Continues the logic inside render_page.

Line 551: <div class="step">1. Upload PDF<small>Scanned books or chapters only.</small></div>

Explanation: Continues the logic inside render_page.

Line 552: <div class="step">2. Automatic processing<small>Up to {max_parallel_jobs()} jobs run together.</small></div>

Explanation: Continues the logic inside render_page.

Line 553: <div class="step">3. Download DOCX<small>Final Word files appear below.</small></div>

Explanation: Continues the logic inside render_page.

Line 554: </div>

Explanation: Continues the logic inside render_page.

Line 555: </section>

Explanation: Continues the logic inside render_page.

Line 556: <section>

Explanation: Continues the logic inside render_page.

Line 557: <h2>Live Progress</h2>

Explanation: Continues the logic inside render_page.

Line 558: <div id="progressBox" class="progressbox {'idle' if running_count == 0 and queued_count == 0 else ''}">

Explanation: Continues the logic inside render_page.

Line 559: <div>

Explanation: Continues the logic inside render_page.

Line 560: <strong id="progressTitle">{'Working on files' if running_count else ('Waiting to start' if queued_count else 'No active jobs')}

Explanation: Continues the logic inside render_page.

Line 561: <small id="progressDetail">Running: {running_count} | Queued: {queued_count} | Completed Word files: {len(completed_docs)}</small>

Explanation: Continues the logic inside render_page.

Line 562: </div>

Explanation: Continues the logic inside render_page.

Line 563: <div id="progressSpinner" class="spinner" style="display: {'block' if running_count or queued_count else 'none'}"></div>

Explanation: Continues the logic inside render_page.

Line 564: </div>

Explanation: Continues the logic inside render_page.

Line 565: </section>

Explanation: Continues the logic inside render_page.

Line 566: <div class="grid">

Explanation: Continues the logic inside render_page.

Line 567: <section>

Explanation: Continues the logic inside render_page.

Line 568: <h2>Upload PDFs</h2>

Explanation: Continues the logic inside render_page.

Line 569: <form id="uploadForm" action="/upload" method="post" enctype="multipart/form-data">

Explanation: Continues the logic inside render_page.

Line 570: <label>Select scanned PDF files</label>

Explanation: Continues the logic inside render_page.

Line 571: <input id="pdfInput" type="file" name="pdfs" accept="application/pdf,.pdf" multiple>

Explanation: Continues the logic inside render_page.

Line 572: <div class="actions"><button id="uploadButton" type="submit">Upload and Start</button>Refresh Status</div>

Explanation: Continues the logic inside render_page.

Line 573: <div class="note">After upload, jobs start automatically. Existing running jobs continue; new uploads wait in the queue.</div>
Explanation: Continues the logic inside render_page.

Line 574: </form>
Explanation: Continues the logic inside render_page.

Line 575: </section>
Explanation: Continues the logic inside render_page.

Line 576: <section>
Explanation: Continues the logic inside render_page.

Line 577: <h2>Processing Settings</h2>
Explanation: Continues the logic inside render_page.

Line 578: <form action="/settings" method="post">
Explanation: Continues the logic inside render_page.

Line 579: <div class="note">Use a page limit for tests only. If a long PDF fails near the end, Run Again / Resume reuses saved verified pages.</div>
Explanation: Continues the logic inside render_page.

Line 580: <label>Structured Nidan Panchak notes</label>
Explanation: Continues the logic inside render_page.

Line 581: <select name="structured">
Explanation: Continues the logic inside render_page.

Line 582: <option value="false" {'selected' if structured_text == 'Off' else ''}>Off: exact book transcription only</option>
Explanation: Continues the logic inside render_page.

Line 583: <option value="true" {'selected' if structured_text == 'On' else ''}>On: also create structured notes</option>
Explanation: Continues the logic inside render_page.

Line 584: </select>
Explanation: Continues the logic inside render_page.

Line 585: <label>Start page</label>
Explanation: Continues the logic inside render_page.

Line 586: <input type="number" name="test_start_page" min="1" value="{test_start_page}">
Explanation: Continues the logic inside render_page.

Line 587: <label>Pages to process</label>
Explanation: Continues the logic inside render_page.

Line 588: <select name="test_max_pages">
Explanation: Continues the logic inside render_page.

Line 589: <option value="0" {'selected' if test_max_pages == '0' else ''}>Whole PDF</option>
Explanation: Continues the logic inside render_page.

Line 590: <option value="1" {'selected' if test_max_pages == '1' else ''}>1 page</option>
Explanation: Continues the logic inside render_page.

Line 591: <option value="2" {'selected' if test_max_pages == '2' else ''}>2 pages</option>
Explanation: Continues the logic inside render_page.

Line 592: <option value="3" {'selected' if test_max_pages == '3' else ''}>3 pages</option>
Explanation: Continues the logic inside render_page.

Line 593: <option value="4" {'selected' if test_max_pages == '4' else ''}>4 pages</option>
Explanation: Continues the logic inside render_page.

Line 594: <option value="5" {'selected' if test_max_pages == '5' else ''}>5 pages</option>
Explanation: Continues the logic inside render_page.

Line 595: </select>
Explanation: Continues the logic inside render_page.

Line 596: <div class="note">Use 1 to 5 pages for tests or free API keys. Choose Whole PDF for full processing.</div>
Explanation: Continues the logic inside render_page.

Line 597: <label>Speed mode</label>
Explanation: Continues the logic inside render_page.

Line 598: <select name="speed_mode">
Explanation: Continues the logic inside render_page.

Line 599: <option value="fast" {'selected' if speed_mode == 'fast' else ''}>Fast: one Gemini pass per page</option>
Explanation: Continues the logic inside render_page.

Line 600: <option value="balanced" {'selected' if speed_mode == 'balanced' else ''}>Balanced: two-pass verification, parallel pages</option>
Explanation: Continues the logic inside render_page.

Line 601: <option value="accuracy" {'selected' if speed_mode == 'accuracy' else ''}>Highest accuracy: slower two-pass verification</option>
Explanation: Continues the logic inside render_page.

Line 602: </select>
Explanation: Continues the logic inside render_page.

Line 603: <label>Page workers per PDF</label>
Explanation: Continues the logic inside render_page.

Line 604: <select name="page_workers">
Explanation: Continues the logic inside render_page.

Line 605: <option value="1" {'selected' if page_workers == '1' else ''}>1 page at a time</option>
Explanation: Continues the logic inside render_page.

Line 606: <option value="2" {'selected' if page_workers == '2' else ''}>2 pages at a time</option>
Explanation: Continues the logic inside render_page.

Line 607: <option value="3" {'selected' if page_workers == '3' else ''}>3 pages at a time</option>
Explanation: Continues the logic inside render_page.

Line 608: <option value="4" {'selected' if page_workers == '4' else ''}>4 pages at a time</option>
Explanation: Continues the logic inside render_page.

Line 609: </select>
Explanation: Continues the logic inside render_page.

Line 610: <label>Maximum parallel jobs</label>
Explanation: Continues the logic inside render_page.

Line 611: <select name="parallel">
Explanation: Continues the logic inside render_page.

Line 612: <option value="1" {'selected' if max_parallel_jobs() == 1 else ''}>1</option>
Explanation: Continues the logic inside render_page.

Line 613: <option value="2" {'selected' if max_parallel_jobs() == 2 else ''}>2</option>
Explanation: Continues the logic inside render_page.

Line 614: <option value="3" {'selected' if max_parallel_jobs() == 3 else ''}>3</option>
Explanation: Continues the logic inside render_page.

Line 615: <option value="5" {'selected' if max_parallel_jobs() == 5 else ''}>5</option>
Explanation: Continues the logic inside render_page.

Line 616: </select>
Explanation: Continues the logic inside render_page.

Line 617: <div class="actions"><button type="submit">Save Settings</button></div>
Explanation: Continues the logic inside render_page.

Line 618: </form>
Explanation: Continues the logic inside render_page.

Line 619: </section>

Explanation: Continues the logic inside render_page.

Line 620: <section class="wide">

Explanation: Continues the logic inside render_page.

Line 621: <h2>Job Queue</h2>

Explanation: Continues the logic inside render_page.

Line 622: <div class="statusbar" style="margin-bottom: 12px;">

Explanation: Continues the logic inside render_page.

Line 623: Running: {running_count}

Explanation: Continues the logic inside render_page.

Line 624: Queued: {queued_count}

Explanation: Continues the logic inside render_page.

Line 625: Total jobs: {len(jobs)}

Explanation: Continues the logic inside render_page.

Line 626: </div>

Explanation: Continues the logic inside render_page.

Line 627: {jobs_html}

Explanation: Continues the logic inside render_page.

Line 628: </section>

Explanation: Continues the logic inside render_page.

Line 629: <section class="wide">

Explanation: Continues the logic inside render_page.

Line 630: <h2>Completed Word Documents</h2>

Explanation: Continues the logic inside render_page.

Line 631: {docs_html}

Explanation: Continues the logic inside render_page.

Line 632: </section>

Explanation: Continues the logic inside render_page.

Line 633: <section class="wide">

Explanation: Continues the logic inside render_page.

Line 634: <h2>Recent Log</h2>

Explanation: Continues the logic inside render_page.

Line 635: <pre id="log">{escape(job_log(jobs[0]['id']) if jobs else 'No jobs yet.')}</pre>

Explanation: Continues the logic inside render_page.

Line 636: </section>

Explanation: Continues the logic inside render_page.

Line 637: </div>

Explanation: Continues the logic inside render_page.

Line 638: </main>

Explanation: Continues the logic inside render_page.

Line 639: <script>

Explanation: Continues the logic inside render_page.

Line 640: let uploadInProgress = false;

Explanation: Continues the logic inside render_page.

Line 641: let lastJobSignature = {json.dumps("|".join(f"{job.get('id')}:{job.get('status')}" for job in jobs))};

Explanation: Converts Python data into JSON text.

Line 642: const uploadForm = document.getElementById('uploadForm');

Explanation: Continues the logic inside render_page.

Line 643: const uploadButton = document.getElementById('uploadButton');

Explanation: Continues the logic inside render_page.

Line 644: const pdfInput = document.getElementById('pdfInput');

Explanation: Continues the logic inside render_page.

Line 645: if (uploadForm) {{

Explanation: Starts a conditional branch; the indented block runs only when this condition is true.

Line 646: uploadForm.addEventListener('submit', (event) => {{

Explanation: Assigns or updates the value of uploadForm.addEventListener('submit', (event)).

Line 647: if (!pdfInput.files || pdfInput.files.length === 0) {{

Explanation: Starts a conditional branch; the indented block runs only when this condition is true.

Line 648: return;

Explanation: Continues the logic inside render_page.

Line 649: }}

Explanation: Continues the logic inside render_page.

Line 650: uploadInProgress = true;

Explanation: Assigns or updates the value of uploadInProgress.

Line 651: uploadButton.disabled = true;

Explanation: Assigns or updates the value of uploadButton.disabled.

Line 652: uploadButton.textContent = 'Uploading...';

Explanation: Assigns or updates the value of uploadButton.textContent.

Line 653: }});

Explanation: Continues the logic inside render_page.

Line 654: }}

Explanation: Continues the logic inside render_page.

Line 655: setInterval(async () => {{

Explanation: Continues the logic inside render_page.

Line 656: if (uploadInProgress) return;

Explanation: Starts a conditional branch; the indented block runs only when this condition is true.

Line 657: const response = await fetch('/status');

Explanation: Continues the logic inside render_page.

Line 658: const data = await response.json();

Explanation: Continues the logic inside render_page.

Line 659: if (data.log) document.getElementById('log').textContent = data.log;

Explanation: Starts a conditional branch; the indented block runs only when this condition is true.

Line 660: const jobs = data.jobs || [];

Explanation: Continues the logic inside render_page.

Line 661: const running = jobs.filter((job) => job.status === 'running').length;

Explanation: Continues the logic inside render_page.

Line 662: const queued = jobs.filter((job) => job.status === 'queued').length;

Explanation: Continues the logic inside render_page.

Line 663: const completed = jobs.filter((job) => job.status === 'completed').length;

Explanation: Continues the logic inside render_page.

Line 664: const failed = jobs.filter((job) => job.status === 'failed').length;

Explanation: Continues the logic inside render_page.

Line 665: const nextJobSignature = jobs.map((job) => `\${job.id}:\${job.status}`).join('|');

Explanation: Continues the logic inside render_page.

Line 666: if (nextJobSignature !== lastJobSignature) {{

Explanation: Starts a conditional branch; the indented block runs only when this condition is true.

Line 667: window.location.reload();

Explanation: Continues the logic inside render_page.

Line 668: return;

Explanation: Continues the logic inside render_page.

Line 669: }})

Explanation: Continues the logic inside render_page.

Line 670: document.getElementById('progressTitle').textContent = running > 0 ? 'Working on files' : (queued > 0 ? 'Waiting to start' : 'No active jobs');

Explanation: Assigns or updates the value of document.getElementById('progressTitle').textContent.

Line 671: document.getElementById('progressDetail').textContent = `Running: \${running} | Queued: \${queued} | Completed jobs: \${completed} | Failed jobs: \${failed}`;

Explanation: Assigns or updates the value of document.getElementById('progressDetail').textContent.

Line 672: document.getElementById('progressSpinner').style.display = (running > 0 || queued > 0) ? 'block' : 'none';

Explanation: Assigns or updates the value of document.getElementById('progressSpinner').style.display.

Line 673: document.getElementById('progressBox').classList.toggle('idle', running === 0 && queued === 0);

Explanation: Assigns or updates the value of document.getElementById('progressBox').classList.toggle('idle', running.

Line 674: }}, 8000);

Explanation: Continues the logic inside render_page.

Line 675: </script>

Explanation: Continues the logic inside render_page.

Line 676: </body>

Explanation: Continues the logic inside render_page.

Line 677: </html>""

Explanation: Continues the logic inside render_page.

Line 678: [blank]

Explanation: Blank line used to separate logical sections.

Line 679: [blank]

Explanation: Blank line used to separate logical sections.

Line 680: @app.get("/")

Explanation: Registers the following function as a Flask route or request hook.

Line 681: def index() -> Response:

Explanation: Serves the main UI.

Line 682: ensure_dispatcher()

Explanation: Continues the logic inside index.

Line 683: return Response(render_page(), mimetype="text/html")

Explanation: Returns the result from index.

Line 684: [blank]

Explanation: Blank line used to separate logical sections.

Line 685: [blank]

Explanation: Blank line used to separate logical sections.

Line 686: @app.get("/health")

Explanation: Registers the following function as a Flask route or request hook.

Line 687: def health() -> Response:

Explanation: Returns a small health-check JSON response.

Line 688: return jsonify({"ok": True, "time": now_text()})

Explanation: Returns the result from health.

Line 689: [blank]

Explanation: Blank line used to separate logical sections.

Line 690: [blank]

Explanation: Blank line used to separate logical sections.

Line 691: @app.post("/upload")

Explanation: Registers the following function as a Flask route or request hook.

Line 692: def upload() -> Response:

Explanation: Handles PDF upload form submissions.

Line 693: `ensure_dispatcher()`

Explanation: Continues the logic inside upload.

Line 694: `uploaded = 0`

Explanation: Assigns or updates the value of uploaded.

Line 695: `rejected: list[str] = []`

Explanation: Assigns or updates the value of rejected: list[str].

Line 696: `for file in request.files.getlist("pdfs"):`

Explanation: Starts a loop over each item in a collection.

Line 697: `if not file or not file.filename:`

Explanation: Starts a conditional branch; the indented block runs only when this condition is true.

Line 698: `continue`

Explanation: Continues the logic inside upload.

Line 699: `filename = safe_pdf_name(file.filename)`

Explanation: Assigns or updates the value of filename.

Line 700: `if not filename.lower().endswith(".pdf"):`

Explanation: Starts a conditional branch; the indented block runs only when this condition is true.

Line 701: `rejected.append(f"{filename}: only PDF files are allowed")`

Explanation: Continues the logic inside upload.

Line 702: `continue`

Explanation: Continues the logic inside upload.

Line 703: `if not looks_like_pdf(file):`

Explanation: Starts a conditional branch; the indented block runs only when this condition is true.

Line 704: `rejected.append(f"{filename}: file is not a readable PDF upload")`

Explanation: Continues the logic inside upload.

Line 705: `continue`

Explanation: Continues the logic inside upload.

Line 706: `if file and file.filename and file.filename.lower().endswith(".pdf"):`

Explanation: Starts a conditional branch; the indented block runs only when this condition is true.

Line 707: `create_job(file)`

Explanation: Continues the logic inside upload.

Line 708: `uploaded += 1`

Explanation: Continues the logic inside upload.

Line 709: `if uploaded == 0:`

Explanation: Starts a conditional branch; the indented block runs only when this condition is true.

Line 710: `reason = "; ".join(rejected) if rejected else "No PDF file was selected. Please choose one or more PDF files."`

Explanation: Assigns or updates the value of reason.

Line 711: `return redirect(url_for("index", error=reason))`

Explanation: Returns the result from upload.

Line 712: `return redirect(url_for("index", uploaded=str(uploaded)))`

Explanation: Returns the result from upload.

Line 713: [blank]

Explanation: Blank line used to separate logical sections.

Line 714: [blank]

Explanation: Blank line used to separate logical sections.

Line 715: `@app.post("/settings")`

Explanation: Registers the following function as a Flask route or request hook.

Line 716: `def settings() -> Response:`

Explanation: Handles processing settings submitted from the UI.

Line 717: structured = request.form.get("structured", "false")

Explanation: Assigns or updates the value of structured.

Line 718: parallel = request.form.get("parallel", "3")

Explanation: Assigns or updates the value of parallel.

Line 719: speed_mode = request.form.get("speed_mode", "balanced")

Explanation: Assigns or updates the value of speed_mode.

Line 720: page_workers = request.form.get("page_workers", "2")

Explanation: Assigns or updates the value of page_workers.

Line 721: if speed_mode not in {"fast", "balanced", "accuracy"}:

Explanation: Starts a conditional branch; the indented block runs only when this condition is true.

Line 722: speed_mode = "accuracy"

Explanation: Assigns or updates the value of speed_mode.

Line 723: test_start_page = parse_int_field(request.form.get("test_start_page"), 1, 1, 10000)

Explanation: Assigns or updates the value of test_start_page.

Line 724: test_max_pages = parse_int_field(request.form.get("test_max_pages"), 0, 0, 5)

Explanation: Assigns or updates the value of test_max_pages.

Line 725: page_workers = parse_int_field(page_workers, 1, 1, 4)

Explanation: Assigns or updates the value of page_workers.

Line 726: parallel = parse_int_field(parallel, 1, 1, 5)

Explanation: Assigns or updates the value of parallel.

Line 727: updates = {"TEST_START_PAGE": test_start_page, "TEST_MAX_PAGES": test_max_pages, "TEST_PDF_LIMIT": "1"}

Explanation: Assigns or updates the value of updates.

Line 728: updates.update(

Explanation: Continues the logic inside settings.

Line 729: {

Explanation: Starts or continues a multi-line expression or data structure.

Line 730: "CREATE_DOCX": "true",

Explanation: Continues the logic inside settings.

Line 731: "CREATE_STRUCTURED_NOTES": "true" if structured == "true" else "false",

Explanation: Continues the logic inside settings.

Line 732: "RESET_MASTER": "false",

Explanation: Continues the logic inside settings.

Line 733: "MAX_PARALLEL_JOBS": parallel,

Explanation: Continues the logic inside settings.

Line 734: "SPEED_MODE": speed_mode,

Explanation: Continues the logic inside settings.

Line 735: "PAGE_WORKERS_PER_JOB": page_workers,

Explanation: Continues the logic inside settings.

Line 736: }

Explanation: Closes a multi-line expression or data structure.

Line 737:)

Explanation: Closes a multi-line expression or data structure.

Line 738: write_env(updates)

Explanation: Continues the logic inside settings.

Line 739: return redirect(url_for("index"))

Explanation: Returns the result from settings.

Line 740: [blank]

Explanation: Blank line used to separate logical sections.

Line 741: [blank]

Explanation: Blank line used to separate logical sections.

Line 742: `@app.post("/retry/<job_id>")`

Explanation: Registers the following function as a Flask route or request hook.

Line 743: `def retry_job(job_id: str) -> Response:`

Explanation: Queues a job for another run.

Line 744: `reset_job(job_id)`

Explanation: Continues the logic inside `retry_job`.

Line 745: `return redirect(url_for("index"))`

Explanation: Returns the result from `retry_job`.

Line 746: [blank]

Explanation: Blank line used to separate logical sections.

Line 747: [blank]

Explanation: Blank line used to separate logical sections.

Line 748: `@app.post("/delete/<job_id>")`

Explanation: Registers the following function as a Flask route or request hook.

Line 749: `def remove_job(job_id: str) -> Response:`

Explanation: Deletes a job from the UI.

Line 750: `delete_job(job_id)`

Explanation: Continues the logic inside `remove_job`.

Line 751: `return redirect(url_for("index"))`

Explanation: Returns the result from `remove_job`.

Line 752: [blank]

Explanation: Blank line used to separate logical sections.

Line 753: [blank]

Explanation: Blank line used to separate logical sections.

Line 754: `@app.get("/status")`

Explanation: Registers the following function as a Flask route or request hook.

Line 755: `def status() -> Response:`

Explanation: Returns current jobs and log text as JSON for the browser refresh script.

Line 756: `jobs = load_jobs()`

Explanation: Assigns or updates the value of `jobs`.

Line 757: `latest_log = job_log(jobs[0]["id"]) if jobs else ""`

Explanation: Assigns or updates the value of `latest_log`.

Line 758: `return jsonify({"refresh": False, "jobs": jobs, "log": latest_log})`

Explanation: Returns the result from `status`.

Line 759: [blank]

Explanation: Blank line used to separate logical sections.

Line 760: [blank]

Explanation: Blank line used to separate logical sections.

Line 761: `@app.get("/download/<job_id>/<path:filename>")`

Explanation: Registers the following function as a Flask route or request hook.

Line 762: `def download_output(job_id: str, filename: str) -> Response:`

Explanation: Downloads a DOCX file from `completed_docs` or a job folder.

Line 763: `if not valid_job_id(job_id):`

Explanation: Starts a conditional branch; the indented block runs only when this condition is true.

Line 764: `return Response("File not found", status=404)`

Explanation: Returns the result from `download_output`.

Line 765: `completed_path = (COMPLETED_DOCS_DIR / Path(filename).name).resolve()`

Explanation: Assigns or updates the value of `completed_path`.

Line 766: `if (`

Explanation: Starts a conditional branch; the indented block runs only when this condition is true.

Line 767: `COMPLETED_DOCS_DIR.resolve()` in `completed_path.parents`

Explanation: Continues the logic inside `download_output`.

Line 768: `and completed_path.suffix.lower() == ".docx"`

Explanation: Assigns or updates the value of `and completed_path.suffix.lower()`.

Line 769: `and completed_path.exists()`

Explanation: Continues the logic inside `download_output`.

Line 770: `)`:

Explanation: Continues the logic inside `download_output`.

Line 771: `return send_file(completed_path, as_attachment=True)`

Explanation: Returns the result from `download_output`.

Line 772: `output_dir = (job_dir(job_id) / "output_notes").resolve()`

Explanation: Assigns or updates the value of `output_dir`.

Line 773: `path = (output_dir / Path(filename).name).resolve()`

Explanation: Assigns or updates the value of `path`.

Line 774: `if output_dir not in path.parents or path.suffix.lower() != ".docx" or not path.exists():`

Explanation: Starts a conditional branch; the indented block runs only when this condition is true.

Line 775: `return Response("File not found", status=404)`

Explanation: Returns the result from `download_output`.

Line 776: `return send_file(path, as_attachment=True)`

Explanation: Returns the result from `download_output`.

Line 777: [blank]

Explanation: Blank line used to separate logical sections.

Line 778: [blank]

Explanation: Blank line used to separate logical sections.

Line 779: `@app.get("/download-doc/<path:filename>")`

Explanation: Registers the following function as a Flask route or request hook.

Line 780: `def download_completed_doc(filename: str) -> Response:`

Explanation: Downloads a DOCX file from `completed_docs`.

Line 781: `path = (COMPLETED_DOCS_DIR / Path(filename).name).resolve()`

Explanation: Assigns or updates the value of `path`.

Line 782: `if COMPLETED_DOCS_DIR.resolve() not in path.parents or path.suffix.lower() != ".docx" or not path.exists():`

Explanation: Starts a conditional branch; the indented block runs only when this condition is true.

Line 783: `return Response("File not found", status=404)`

Explanation: Returns the result from `download_completed_doc`.

Line 784: `return send_file(path, as_attachment=True)`

Explanation: Returns the result from `download_completed_doc`.

Line 785: [blank]

Explanation: Blank line used to separate logical sections.

Line 786: [blank]

Explanation: Blank line used to separate logical sections.

Line 787: `if __name__ == "__main__":`

Explanation: Starts a conditional branch; the indented block runs only when this condition is true.

Line 788: `ensure_project_files()`

Explanation: Continues the logic inside `download_completed_doc`.

Line 789: `ensure_dispatcher()`

Explanation: Continues the logic inside `download_completed_doc`.

Line 790: `host = os.getenv("HOST", "127.0.0.1")`

Explanation: Assigns or updates the value of `host`.

Line 791: `port = int(os.getenv("PORT", "7860"))`

Explanation: Assigns or updates the value of `port`.

Line 792: `app.run(host=host, port=port, debug=False, threaded=True)`

Explanation: Assigns or updates the value of `app.run(host`.

ultimate_book_processor.py

PDF-to-Gemini processing pipeline, page resume state, and DOCX output.

Source path: C:\Users\sawan\Desktop\new_project\Ayurvedic_Book_Processor\ultimate_book_processor.py

Line count: 957

Line 1: from __future__ import annotations

Explanation: Enables future Python behavior for type annotations.

Line 2: [blank]

Explanation: Blank line used to separate logical sections.

Line 3: import io

Explanation: Imports a Python module used later in this file.

Line 4: import json

Explanation: Imports a Python module used later in this file.

Line 5: import logging

Explanation: Imports a Python module used later in this file.

Line 6: import os

Explanation: Imports a Python module used later in this file.

Line 7: import sys

Explanation: Imports a Python module used later in this file.

Line 8: import time

Explanation: Imports a Python module used later in this file.

Line 9: import traceback

Explanation: Imports a Python module used later in this file.

Line 10: import threading

Explanation: Imports a Python module used later in this file.

Line 11: from concurrent.futures import ThreadPoolExecutor, as_completed

Explanation: Imports specific classes or functions from another module.

Line 12: from dataclasses import dataclass

Explanation: Imports specific classes or functions from another module.

Line 13: from datetime import datetime, timezone

Explanation: Imports specific classes or functions from another module.

Line 14: from pathlib import Path

Explanation: Imports specific classes or functions from another module.

Line 15: from typing import Any

Explanation: Imports specific classes or functions from another module.

Line 16: [blank]

Explanation: Blank line used to separate logical sections.

Line 17: from dotenv import load_dotenv

Explanation: Imports specific classes or functions from another module.

Line 18: from google import genai

Explanation: Imports specific classes or functions from another module.

Line 19: from google.genai import types

Explanation: Imports specific classes or functions from another module.

Line 20: from pdf2image import convert_from_path, pdfinfo_from_path

Explanation: Imports specific classes or functions from another module.

Line 21: from pdf2image.exceptions import PDFInfoNotInstalledError, PDFPageCountError, PDFSyntaxError

Explanation: Imports specific classes or functions from another module.

Line 22: from PIL import Image

Explanation: Imports specific classes or functions from another module.

Line 23: [blank]

Explanation: Blank line used to separate logical sections.

Line 24: [blank]

Explanation: Blank line used to separate logical sections.

Line 25: EXTRACTION_PROMPT = (

Explanation: Assigns or updates the value of EXTRACTION_PROMPT.

Line 26: "You are reading a scanned Ayurvedic medical textbook page. Transcribe the visible text as accurately as possible. "

Explanation: Continues the logic inside this file.

Line 27: "Preserve Hindi, Sanskrit, English, headings, footnotes, tables, figure captions, page numbers, and medical terms. "

Explanation: Continues the logic inside this file.

Line 28: "Preserve the page's visible structure as much as possible, including line breaks, section order, lists, table layout, and mixed-language formatting. "

Explanation: Continues the logic inside this file.

Line 29: "Do not force this page into any fixed template. "

Explanation: Continues the logic inside this file.

Line 30: "Do not summarize. Do not translate unless translation is already printed on the page. If a word is unclear, "

Explanation: Continues the logic inside this file.

Line 31: "write [unclear]. Do not guess. Return only the transcription."

Explanation: Continues the logic inside this file.

Line 32:)

Explanation: Closes a multi-line expression or data structure.

Line 33: [blank]

Explanation: Blank line used to separate logical sections.

Line 34: VERIFICATION_PROMPT = (

Explanation: Assigns or updates the value of VERIFICATION_PROMPT.

Line 35: "You are a strict medical textbook transcription verifier. Compare the extracted text against the scanned page image. "

Explanation: Continues the logic inside this file.

Line 36: "Correct only mistakes that are visible in the image. Do not add missing information from memory. Do not summarize. "

Explanation: Continues the logic inside this file.

Line 37: "Preserve Hindi, Sanskrit, English, headings, tables, footnotes, figure captions, and page order. If text is unreadable, "

Explanation: Continues the logic inside this file.

Line 38: "mark it as [unclear]. Preserve the original page's visible structure as much as possible. "

Explanation: Continues the logic inside this file.

Line 39: "Do not force this page into any fixed template. Return only the corrected verified transcription."

Explanation: Continues the logic inside this file.

Line 40:)

Explanation: Closes a multi-line expression or data structure.

Line 41: [blank]

Explanation: Blank line used to separate logical sections.

Line 42: FORMAT_PROMPT = ""You are converting verified Ayurvedic textbook transcription into structured study notes. Use only the verified source text. Do not invent anything. Do not add modern medical explanation unless it is present in the source. Preserve Ayurvedic terms. If a section is not clearly present, write: Not clearly mentioned in source text.

Explanation: Assigns or updates the value of FORMAT_PROMPT.

Line 43: Preserve the source languages exactly: Hindi should remain Hindi, Sanskrit should remain Sanskrit, and English should remain English. Do not translate Hindi or Sanskrit into English. Do not romanize Devanagari unless the verified source text itself is romanized.

Explanation: Continues the logic inside this file.

Line 44: [blank]

Explanation: Blank line used to separate logical sections.

Line 45: Required Markdown format:

Explanation: Continues the logic inside this file.

Line 46: [blank]

Explanation: Blank line used to separate logical sections.

Line 47: # Chapter: <chapter title>

Explanation: Comment for maintainers; Python ignores it at runtime.

Line 48: [blank]

Explanation: Blank line used to separate logical sections.

Line 49: ## Source PDF

Explanation: Comment for maintainers; Python ignores it at runtime.

Line 50: <filename>

Explanation: Continues the logic inside this file.

Line 51: [blank]

Explanation: Blank line used to separate logical sections.

Line 52: ## Verified Source Pages

Explanation: Comment for maintainers; Python ignores it at runtime.

Line 53: Pages: <start page> to <end page>

Explanation: Continues the logic inside this file.

Line 54: [blank]

Explanation: Blank line used to separate logical sections.

Line 55: ## Nidan Panchak Notes

Explanation: Comment for maintainers; Python ignores it at runtime.

Line 56: [blank]

Explanation: Blank line used to separate logical sections.

Line 57: ### 1. Nidana - Causes / Etiology

Explanation: Comment for maintainers; Python ignores it at runtime.

Line 58: - ...

Explanation: Continues the logic inside this file.

Line 59: [blank]

Explanation: Blank line used to separate logical sections.

Line 60: ### 2. Purvarupa - Premonitory Symptoms

Explanation: Comment for maintainers; Python ignores it at runtime.

Line 61: - ...

Explanation: Continues the logic inside this file.

Line 62: [blank]

Explanation: Blank line used to separate logical sections.

Line 63: ### 3. Rupa - Signs and Symptoms

Explanation: Comment for maintainers; Python ignores it at runtime.

Line 64: - ...

Explanation: Continues the logic inside this file.

Line 65: [blank]

Explanation: Blank line used to separate logical sections.

Line 66: ### 4. Upashaya / Anupashaya - Relieving and Aggravating Factors

Explanation: Comment for maintainers; Python ignores it at runtime.

Line 67: - ...

Explanation: Continues the logic inside this file.

Line 68: [blank]

Explanation: Blank line used to separate logical sections.

Line 69: ### 5. Samprapti - Pathogenesis

Explanation: Comment for maintainers; Python ignores it at runtime.

Line 70: - ...

Explanation: Continues the logic inside this file.

Line 71: [blank]

Explanation: Blank line used to separate logical sections.

Line 72: ## Disease-wise Notes

Explanation: Comment for maintainers; Python ignores it at runtime.

Line 73: [blank]

Explanation: Blank line used to separate logical sections.

Line 74: For every disease/disorder mentioned in the source, create:

Explanation: Continues the logic inside this file.

Line 75: [blank]

Explanation: Blank line used to separate logical sections.

Line 76: ### <Disease / Disorder Name>

Explanation: Comment for maintainers; Python ignores it at runtime.

Line 77: [blank]

Explanation: Blank line used to separate logical sections.

Line 78: ##### Definition / Description

Explanation: Comment for maintainers; Python ignores it at runtime.

Line 79: - ...

Explanation: Continues the logic inside this file.

Line 80: [blank]

Explanation: Blank line used to separate logical sections.

Line 81: ##### Nidana

Explanation: Comment for maintainers; Python ignores it at runtime.

Line 82: - ...

Explanation: Continues the logic inside this file.

Line 83: [blank]

Explanation: Blank line used to separate logical sections.

Line 84: ##### Purvarupa

Explanation: Comment for maintainers; Python ignores it at runtime.

Line 85: - ...

Explanation: Continues the logic inside this file.

Line 86: [blank]

Explanation: Blank line used to separate logical sections.

Line 87: ##### Rupa

Explanation: Comment for maintainers; Python ignores it at runtime.

Line 88: - ...

Explanation: Continues the logic inside this file.

Line 89: [blank]

Explanation: Blank line used to separate logical sections.

Line 90: ##### Upashaya / Anupashaya

Explanation: Comment for maintainers; Python ignores it at runtime.

Line 91: - ...

Explanation: Continues the logic inside this file.

Line 92: [blank]

Explanation: Blank line used to separate logical sections.

Line 93: ##### Samprapti

Explanation: Comment for maintainers; Python ignores it at runtime.

Line 94: - ...

Explanation: Continues the logic inside this file.

Line 95: [blank]

Explanation: Blank line used to separate logical sections.

Line 96: ##### Commentary Notes

Explanation: Comment for maintainers; Python ignores it at runtime.

Line 97: - ...

Explanation: Continues the logic inside this file.

Line 98: [blank]

Explanation: Blank line used to separate logical sections.

Line 99: ##### Important Terms

Explanation: Comment for maintainers; Python ignores it at runtime.

Line 100: - ...

Explanation: Continues the logic inside this file.

Line 101: [blank]

Explanation: Blank line used to separate logical sections.

Line 102: ##### Unclear / Needs Review

Explanation: Comment for maintainers; Python ignores it at runtime.

Line 103: - ...

Explanation: Continues the logic inside this file.

Line 104: [blank]

Explanation: Blank line used to separate logical sections.

Line 105: ## Important Sanskrit / Ayurvedic Terms

Explanation: Comment for maintainers; Python ignores it at runtime.

Line 106: [blank]

Explanation: Blank line used to separate logical sections.

Line 107: | Term | Meaning | Context |

Explanation: Continues the logic inside this file.

Line 108: |-----|-----|-----|

Explanation: Continues the logic inside this file.

Line 109: [blank]

Explanation: Blank line used to separate logical sections.

Line 110: ## Tables Found in Source

Explanation: Comment for maintainers; Python ignores it at runtime.

Line 111: Recreate important source tables in Markdown only if present in the verified text.

Explanation: Continues the logic inside this file.

Line 112: [blank]

Explanation: Blank line used to separate logical sections.

Line 113: ## Figures / Diagrams Found in Source

Explanation: Comment for maintainers; Python ignores it at runtime.

Line 114: Mention:

Explanation: Continues the logic inside this file.

Line 115: - figure number

Explanation: Continues the logic inside this file.

Line 116: - page number

Explanation: Continues the logic inside this file.

Line 117: - caption

Explanation: Continues the logic inside this file.

Line 118: - what the figure shows

Explanation: Continues the logic inside this file.

Line 119: [blank]

Explanation: Blank line used to separate logical sections.

Line 120: ## Key Clinical Takeaways

Explanation: Comment for maintainers; Python ignores it at runtime.

Line 121: - ...

Explanation: Continues the logic inside this file.

Line 122: [blank]

Explanation: Blank line used to separate logical sections.

Line 123: ## Unclear / Needs Human Review

Explanation: Comment for maintainers; Python ignores it at runtime.

Line 124: - ...

Explanation: Continues the logic inside this file.

Line 125: ""

Explanation: Continues the logic inside this file.

Line 126: [blank]

Explanation: Blank line used to separate logical sections.

Line 127: [blank]

Explanation: Blank line used to separate logical sections.

Line 128: STATE_IO_LOCK = threading.Lock()

Explanation: Assigns or updates the value of STATE_IO_LOCK.

Line 129: [blank]

Explanation: Blank line used to separate logical sections.

Line 130: [blank]

Explanation: Blank line used to separate logical sections.

Line 131: @dataclass(frozen=True)

Explanation: Turns the following class into a data container with generated methods.

Line 132: class Config:

Explanation: Defines the Config class.

Line 133: gemini_api_key: str

Explanation: Continues the logic inside this file.

Line 134: gemini_model: str

Explanation: Continues the logic inside this file.

Line 135: pdf_dir: Path

Explanation: Continues the logic inside this file.

Line 136: output_dir: Path

Explanation: Continues the logic inside this file.

Line 137: poppler_path: str | None

Explanation: Continues the logic inside this file.

Line 138: dpi: int

Explanation: Continues the logic inside this file.

Line 139: max_retries: int

Explanation: Continues the logic inside this file.

Line 140: gemini_delay_seconds: float

Explanation: Continues the logic inside this file.

Line 141: gemini_timeout_seconds: int

Explanation: Continues the logic inside this file.

Line 142: speed_mode: str

Explanation: Continues the logic inside this file.

Line 143: page_workers_per_job: int

Explanation: Continues the logic inside this file.

Line 144: reset_master: bool

Explanation: Continues the logic inside this file.

Line 145: create_docx: bool

Explanation: Continues the logic inside this file.

Line 146: create_structured_notes: bool

Explanation: Continues the logic inside this file.

Line 147: test_start_page: int

Explanation: Continues the logic inside this file.

Line 148: test_max_pages: int

Explanation: Continues the logic inside this file.

Line 149: test_pdf_limit: int

Explanation: Continues the logic inside this file.

Line 150: root_dir: Path

Explanation: Continues the logic inside this file.

Line 151: extracted_dir: Path

Explanation: Continues the logic inside this file.

Line 152: verified_dir: Path

Explanation: Continues the logic inside this file.

Line 153: chapter_sources_dir: Path

Explanation: Continues the logic inside this file.

Line 154: logs_dir: Path

Explanation: Continues the logic inside this file.

Line 155: state_file: Path

Explanation: Continues the logic inside this file.

Line 156: master_notes_file: Path

Explanation: Continues the logic inside this file.

Line 157: [blank]

Explanation: Blank line used to separate logical sections.

Line 158: [blank]

Explanation: Blank line used to separate logical sections.

Line 159: def utc_now() -> str:

Explanation: Returns the current UTC timestamp for processing state.

Line 160: return datetime.now(timezone.utc).isoformat()

Explanation: Returns the result from utc_now.

Line 161: [blank]

Explanation: Blank line used to separate logical sections.

Line 162: [blank]

Explanation: Blank line used to separate logical sections.

Line 163: def parse_bool(value: str | None, default: bool = False) -> bool:

Explanation: Converts common text values into True or False.

Line 164: if value is None:

Explanation: Starts a conditional branch; the indented block runs only when this condition is true.

Line 165: return default

Explanation: Returns the result from parse_bool.

Line 166: return value.strip().lower() in {"1", "true", "yes", "y", "on"}

Explanation: Returns the result from parse_bool.

Line 167: [blank]

Explanation: Blank line used to separate logical sections.

Line 168: [blank]

Explanation: Blank line used to separate logical sections.

Line 169: `def parse_int(value: str | None, default: int, minimum: int = 1) -> int:`

Explanation: Converts text to an integer with a minimum value.

Line 170: `try:`

Explanation: Starts a protected block where errors can be caught.

Line 171: `parsed = int(str(value).strip()) if value is not None else default`

Explanation: Assigns or updates the value of `parsed`.

Line 172: `except ValueError:`

Explanation: Handles an error raised inside the matching try block.

Line 173: `return default`

Explanation: Returns the result from `parse_int`.

Line 174: `return max(parsed, minimum)`

Explanation: Returns the result from `parse_int`.

Line 175: [blank]

Explanation: Blank line used to separate logical sections.

Line 176: [blank]

Explanation: Blank line used to separate logical sections.

Line 177: `def parse_float(value: str | None, default: float, minimum: float = 0.0) -> float:`

Explanation: Converts text to a float with a minimum value.

Line 178: `try:`

Explanation: Starts a protected block where errors can be caught.

Line 179: `parsed = float(str(value).strip()) if value is not None else default`

Explanation: Assigns or updates the value of `parsed`.

Line 180: `except ValueError:`

Explanation: Handles an error raised inside the matching try block.

Line 181: `return default`

Explanation: Returns the result from `parse_float`.

Line 182: `return max(parsed, minimum)`

Explanation: Returns the result from `parse_float`.

Line 183: [blank]

Explanation: Blank line used to separate logical sections.

Line 184: [blank]

Explanation: Blank line used to separate logical sections.

Line 185: `def load_config() -> Config:`

Explanation: Reads all processor settings and converts paths into absolute paths.

Line 186: `load_dotenv()`

Explanation: Continues the logic inside `load_config`.

Line 187: `root_dir = Path.cwd()`

Explanation: Assigns or updates the value of `root_dir`.

Line 188: `pdf_dir = Path(os.getenv("PDF_DIR", "./pdfs")).expanduser()`

Explanation: Assigns or updates the value of `pdf_dir`.

Line 189: `output_dir = Path(os.getenv("OUTPUT_DIR", "./output_notes")).expanduser()`

Explanation: Assigns or updates the value of `output_dir`.

Line 190: [blank]

Explanation: Blank line used to separate logical sections.

Line 191: `if not pdf_dir.is_absolute():`

Explanation: Starts a conditional branch; the indented block runs only when this condition is true.

Line 192: `pdf_dir = root_dir / pdf_dir`

Explanation: Assigns or updates the value of `pdf_dir`.

Line 193: `if not output_dir.is_absolute():`

Explanation: Starts a conditional branch; the indented block runs only when this condition is true.

Line 194: `output_dir = root_dir / output_dir`

Explanation: Assigns or updates the value of `output_dir`.

Line 195: [blank]

Explanation: Blank line used to separate logical sections.

Line 196: `poppler_path = os.getenv("POPPLER_PATH")`

Explanation: Assigns or updates the value of `poppler_path`.

Line 197: `if poppler_path:`

Explanation: Starts a conditional branch; the indented block runs only when this condition is true.

Line 198: `poppler_path = poppler_path.strip().strip('') or None`

Explanation: Assigns or updates the value of `poppler_path`.

Line 199: [blank]

Explanation: Blank line used to separate logical sections.

Line 200: `speed_mode = (os.getenv("SPEED_MODE", "accuracy").strip().lower() or "accuracy")`

Explanation: Assigns or updates the value of `speed_mode`.

Line 201: `if speed_mode not in {"fast", "balanced", "accuracy"}:`

Explanation: Starts a conditional branch; the indented block runs only when this condition is true.

Line 202: `speed_mode = "accuracy"`

Explanation: Assigns or updates the value of `speed_mode`.

Line 203: [blank]

Explanation: Blank line used to separate logical sections.

Line 204: `return Config(`

Explanation: Returns the result from `load_config`.

Line 205: `gemi_api_key=os.getenv("GEMINI_API_KEY", "").strip(),`

Explanation: Assigns or updates the value of `gemi_api_key`.

Line 206: `gemi_model=os.getenv("GEMINI_MODEL", "gemini-2.5-flash").strip() or "gemini-2.5-flash",`

Explanation: Assigns or updates the value of `gemi_model`.

Line 207: `pdf_dir=pdf_dir,`

Explanation: Assigns or updates the value of `pdf_dir`.

Line 208: `output_dir=output_dir,`

Explanation: Assigns or updates the value of `output_dir`.

Line 209: `poppler_path=poppler_path,`

Explanation: Assigns or updates the value of `poppler_path`.

Line 210: `dpi=parse_int(os.getenv("DPI"), 250, minimum=72),`

Explanation: Assigns or updates the value of `dpi`.

Line 211: `max_retries=parse_int(os.getenv("MAX_RETRIES"), 5, minimum=1),`

Explanation: Assigns or updates the value of `max_retries`.

Line 212: `gemi_delay_seconds=parse_float(os.getenv("GEMINI_DELAY_SECONDS"), 5.0, minimum=0.0),`

Explanation: Assigns or updates the value of `gemi_delay_seconds`.

Line 213: `gemi_timeout_seconds=parse_int(os.getenv("GEMINI_TIMEOUT_SECONDS"), 180, minimum=30),`

Explanation: Assigns or updates the value of `gemi_timeout_seconds`.

Line 214: `speed_mode=speed_mode,`

Explanation: Assigns or updates the value of `speed_mode`.

Line 215: `page_workers_per_job=parse_int(os.getenv("PAGE_WORKERS_PER_JOB"), 1, minimum=1),`

Explanation: Assigns or updates the value of `page_workers_per_job`.

Line 216: `reset_master=parse_bool(os.getenv("RESET_MASTER"), default=False),`

Explanation: Assigns or updates the value of `reset_master`.

Line 217: `create_docx=parse_bool(os.getenv("CREATE_DOCX"), default=False),`

Explanation: Assigns or updates the value of `create_docx`.

Line 218: `create_structured_notes=parse_bool(os.getenv("CREATE_STRUCTURED_NOTES"), default=False),`

Explanation: Assigns or updates the value of `create_structured_notes`.

Line 219: test_start_page=parse_int(os.getenv("TEST_START_PAGE"), 1, minimum=1),
Explanation: Assigns or updates the value of test_start_page.

Line 220: test_max_pages=parse_int(os.getenv("TEST_MAX_PAGES"), 0, minimum=0),
Explanation: Assigns or updates the value of test_max_pages.

Line 221: test_pdf_limit=parse_int(os.getenv("TEST_PDF_LIMIT"), 0, minimum=0),
Explanation: Assigns or updates the value of test_pdf_limit.

Line 222: root_dir=root_dir,
Explanation: Assigns or updates the value of root_dir.

Line 223: extracted_dir=root_dir / "extracted_pages",
Explanation: Assigns or updates the value of extracted_dir.

Line 224: verified_dir=root_dir / "verified_pages",
Explanation: Assigns or updates the value of verified_dir.

Line 225: chapter_sources_dir=root_dir / "chapter_sources",
Explanation: Assigns or updates the value of chapter_sources_dir.

Line 226: logs_dir=root_dir / "logs",
Explanation: Assigns or updates the value of logs_dir.

Line 227: state_file=root_dir / "processing_state.json",
Explanation: Assigns or updates the value of state_file.

Line 228: master_notes_file=root_dir / "Complete_Book_Notes.md",
Explanation: Assigns or updates the value of master_notes_file.

Line 229:)
Explanation: Closes a multi-line expression or data structure.

Line 230: [blank]
Explanation: Blank line used to separate logical sections.

Line 231: [blank]
Explanation: Blank line used to separate logical sections.

Line 232: def setup_logging(config: Config) -> None:
Explanation: Configures logging to both processor.log and the console.

Line 233: config.logs_dir.mkdir(parents=True, exist_ok=True)
Explanation: Creates a folder if it does not already exist.

Line 234: logging.basicConfig(
Explanation: Writes diagnostic information into the processor log.

Line 235: level=logging.INFO,
Explanation: Writes diagnostic information into the processor log.

Line 236: format="%(asctime)s [%(levelname)s] %(message)s",
Explanation: Assigns or updates the value of format.

Line 237: handlers=[
Explanation: Assigns or updates the value of handlers.

Line 238: logging.FileHandler(config.logs_dir / "processor.log", encoding="utf-8"),
Explanation: Writes diagnostic information into the processor log.

Line 239: logging.StreamHandler(sys.stdout),
Explanation: Writes diagnostic information into the processor log.

Line 240:],
Explanation: Continues the logic inside setup_logging.

Line 241:)
Explanation: Closes a multi-line expression or data structure.

Line 242: [blank]
Explanation: Blank line used to separate logical sections.

Line 243: [blank]
Explanation: Blank line used to separate logical sections.

Line 244: `def default_state() -> dict[str, Any]:`

Explanation: Creates the default processing_state.json structure.

Line 245: `return {`

Explanation: Returns the result from default_state.

Line 246: `"created_at": utc_now(),`

Explanation: Continues the logic inside default_state.

Line 247: `"updated_at": utc_now(),`

Explanation: Continues the logic inside default_state.

Line 248: `"completed_pdfs": [],`

Explanation: Continues the logic inside default_state.

Line 249: `"completed_pages": {},`

Explanation: Continues the logic inside default_state.

Line 250: `"extracted_pages": {},`

Explanation: Continues the logic inside default_state.

Line 251: `"verified_pages": {},`

Explanation: Continues the logic inside default_state.

Line 252: `"failed_pages": {},`

Explanation: Continues the logic inside default_state.

Line 253: `"formatted_chapters": [],`

Explanation: Continues the logic inside default_state.

Line 254: `"retry_counts": {},`

Explanation: Continues the logic inside default_state.

Line 255: `"chapter_page_counts": {},`

Explanation: Continues the logic inside default_state.

Line 256: `"master_appended_chapters": [],`

Explanation: Continues the logic inside default_state.

Line 257: `}`

Explanation: Closes a multi-line expression or data structure.

Line 258: [blank]

Explanation: Blank line used to separate logical sections.

Line 259: [blank]

Explanation: Blank line used to separate logical sections.

Line 260: `def normalize_state(state: dict[str, Any]) -> dict[str, Any]:`

Explanation: Repairs missing or wrong-shaped state fields after loading JSON.

Line 261: `base = default_state()`

Explanation: Assigns or updates the value of base.

Line 262: `for key, value in state.items():`

Explanation: Starts a loop over each item in a collection.

Line 263: `base[key] = value`

Explanation: Assigns or updates the value of base[key].

Line 264: `for key in [`

Explanation: Starts a loop over each item in a collection.

Line 265: `"completed_pdfs",`

Explanation: Continues the logic inside normalize_state.

Line 266: `"formatted_chapters",`

Explanation: Continues the logic inside normalize_state.

Line 267: `"master_appended_chapters",`

Explanation: Continues the logic inside normalize_state.

Line 268: `]:`

Explanation: Continues the logic inside normalize_state.

Line 269: if not isinstance(base.get(key), list):

Explanation: Starts a conditional branch; the indented block runs only when this condition is true.

Line 270: base[key] = []

Explanation: Assigns or updates the value of base[key].

Line 271: for key in [

Explanation: Starts a loop over each item in a collection.

Line 272: "completed_pages",

Explanation: Continues the logic inside normalize_state.

Line 273: "extracted_pages",

Explanation: Continues the logic inside normalize_state.

Line 274: "verified_pages",

Explanation: Continues the logic inside normalize_state.

Line 275: "failed_pages",

Explanation: Continues the logic inside normalize_state.

Line 276: "retry_counts",

Explanation: Continues the logic inside normalize_state.

Line 277: "chapter_page_counts",

Explanation: Continues the logic inside normalize_state.

Line 278:]:

Explanation: Continues the logic inside normalize_state.

Line 279: if not isinstance(base.get(key), dict):

Explanation: Starts a conditional branch; the indented block runs only when this condition is true.

Line 280: base[key] = {}

Explanation: Assigns or updates the value of base[key].

Line 281: return base

Explanation: Returns the result from normalize_state.

Line 282: [blank]

Explanation: Blank line used to separate logical sections.

Line 283: [blank]

Explanation: Blank line used to separate logical sections.

Line 284: def load_state(config: Config) -> dict[str, Any]:

Explanation: Loads processing_state.json and backs up invalid JSON if needed.

Line 285: if not config.state_file.exists():

Explanation: Starts a conditional branch; the indented block runs only when this condition is true.

Line 286: return default_state()

Explanation: Returns the result from load_state.

Line 287: try:

Explanation: Starts a protected block where errors can be caught.

Line 288: with config.state_file.open("r", encoding="utf-8") as handle:

Explanation: Opens a context-managed resource and closes it automatically afterward.

Line 289: return normalize_state(json.load(handle))

Explanation: Returns the result from load_state.

Line 290: except json.JSONDecodeError:

Explanation: Handles an error raised inside the matching try block.

Line 291: backup = config.state_file.with_suffix(f".invalid_{int(time.time())}.json")

Explanation: Assigns or updates the value of backup.

Line 292: config.state_file.replace(backup)

Explanation: Continues the logic inside load_state.

Line 293: logging.error("Invalid processing_state.json moved to %s. Starting with a fresh state.", backup)

Explanation: Writes diagnostic information into the processor log.

Line 294: return default_state()

Explanation: Returns the result from load_state.

Line 295: except Exception:

Explanation: Handles an error raised inside the matching try block.

Line 296: logging.error("Could not load processing_state.json:\n%s", traceback.format_exc())

Explanation: Writes diagnostic information into the processor log.

Line 297: return default_state()

Explanation: Returns the result from load_state.

Line 298: [blank]

Explanation: Blank line used to separate logical sections.

Line 299: [blank]

Explanation: Blank line used to separate logical sections.

Line 300: def save_state(config: Config, state: dict[str, Any]) -> None:

Explanation: Writes processing_state.json safely.

Line 301: with STATE_IO_LOCK:

Explanation: Opens a context-managed resource and closes it automatically afterward.

Line 302: state["updated_at"] = utc_now()

Explanation: Assigns or updates the value of state["updated_at"].

Line 303: temp_file = config.state_file.with_suffix(".json.tmp")

Explanation: Assigns or updates the value of temp_file.

Line 304: with temp_file.open("w", encoding="utf-8") as handle:

Explanation: Opens a context-managed resource and closes it automatically afterward.

Line 305: json.dump(state, handle, ensure_ascii=False, indent=2)

Explanation: Converts Python data into JSON text.

Line 306: temp_file.replace(config.state_file)

Explanation: Continues the logic inside save_state.

Line 307: [blank]

Explanation: Blank line used to separate logical sections.

Line 308: [blank]

Explanation: Blank line used to separate logical sections.

Line 309: def ensure_output_dirs(config: Config) -> None:

Explanation: Creates all folders needed by the processor.

Line 310: for directory in [

Explanation: Starts a loop over each item in a collection.

Line 311: config.pdf_dir,

Explanation: Continues the logic inside ensure_output_dirs.

Line 312: config.output_dir,

Explanation: Continues the logic inside ensure_output_dirs.

Line 313: config.extracted_dir,

Explanation: Continues the logic inside ensure_output_dirs.

Line 314: config.verified_dir,

Explanation: Continues the logic inside ensure_output_dirs.

Line 315: config.chapter_sources_dir,

Explanation: Continues the logic inside ensure_output_dirs.

Line 316: config.logs_dir,

Explanation: Continues the logic inside ensure_output_dirs.

Line 317:]:

Explanation: Continues the logic inside ensure_output_dirs.

Line 318: directory.mkdir(parents=True, exist_ok=True)

Explanation: Creates a folder if it does not already exist.

Line 319: [blank]

Explanation: Blank line used to separate logical sections.

Line 320: [blank]

Explanation: Blank line used to separate logical sections.

Line 321: `def find_pdfs(config: Config) -> list[Path]:`

Explanation: Finds PDF files to process.

Line 322: `if not config.pdf_dir.exists():`

Explanation: Starts a conditional branch; the indented block runs only when this condition is true.

Line 323: `config.pdf_dir.mkdir(parents=True, exist_ok=True)`

Explanation: Creates a folder if it does not already exist.

Line 324: `print(f"Created missing PDF folder: {config.pdf_dir}")`

Explanation: Writes progress text to the console or job log.

Line 325: `print("Put scanned textbook PDF files in this folder, then run the script again.")`

Explanation: Writes progress text to the console or job log.

Line 326: `logging.warning("PDF folder was missing and has been created: %s", config.pdf_dir)`

Explanation: Writes diagnostic information into the processor log.

Line 327: `return []`

Explanation: Returns the result from find_pdfs.

Line 328: `pdfs = sorted([path for path in config.pdf_dir.iterdir() if path.is_file() and path.suffix.lower() == ".pdf"])`

Explanation: Assigns or updates the value of pdfs.

Line 329: `skipped = sorted([path.name for path in config.pdf_dir.iterdir() if path.is_file() and path.suffix.lower() != ".pdf"])`

Explanation: Assigns or updates the value of skipped.

Line 330: `for name in skipped:`

Explanation: Starts a loop over each item in a collection.

Line 331: `logging.info("Skipping non-PDF file in pdfs folder: %s", name)`

Explanation: Writes diagnostic information into the processor log.

Line 332: `return pdfs`

Explanation: Returns the result from find_pdfs.

Line 333: [blank]

Explanation: Blank line used to separate logical sections.

Line 334: [blank]

Explanation: Blank line used to separate logical sections.

Line 335: `def pdf2image_kwargs(config: Config) -> dict[str, Any]:`

Explanation: Builds arguments for pdf2image, including Poppler path.

Line 336: `kwargs: dict[str, Any] = {"dpi": config.dpi}`

Explanation: Assigns or updates the value of kwargs: dict[str, Any].

Line 337: `if config.poppler_path:`

Explanation: Starts a conditional branch; the indented block runs only when this condition is true.

Line 338: `kwargs["poppler_path"] = config.poppler_path`

Explanation: Assigns or updates the value of kwargs["poppler_path"].

Line 339: `return kwargs`

Explanation: Returns the result from pdf2image_kwargs.

Line 340: [blank]

Explanation: Blank line used to separate logical sections.

Line 341: [blank]

Explanation: Blank line used to separate logical sections.

Line 342: `def get_pdf_page_count(pdf_path: Path, config: Config) -> int:`

Explanation: Reads the number of pages in a PDF.

Line 343: kwargs = pdf2image_kwargs(config)

Explanation: Assigns or updates the value of kwargs.

Line 344: kwargs.pop("dpi", None)

Explanation: Continues the logic inside get_pdf_page_count.

Line 345: try:

Explanation: Starts a protected block where errors can be caught.

Line 346: info = pdfinfo_from_path(str(pdf_path), **kwargs)

Explanation: Assigns or updates the value of info.

Line 347: return int(info.get("Pages", 0))

Explanation: Returns the result from get_pdf_page_count.

Line 348: except PDFInfoNotInstalledError as exc:

Explanation: Handles an error raised inside the matching try block.

Line 349: message = (

Explanation: Assigns or updates the value of message.

Line 350: "Poppler is not installed or POPPLER_PATH is wrong. On Windows, install Poppler and set "

Explanation: Continues the logic inside get_pdf_page_count.

Line 351: "POPPLER_PATH=C:\\poppler\\bin in .env, then run again."

Explanation: Continues the logic inside get_pdf_page_count.

Line 352:)

Explanation: Closes a multi-line expression or data structure.

Line 353: raise RuntimeError(message) from exc

Explanation: Raises an error so the caller knows processing cannot continue normally.

Line 354: except (PDFPageCountError, PDFSyntaxError) as exc:

Explanation: Handles an error raised inside the matching try block.

Line 355: raise RuntimeError(f"Could not read PDF pages. The file may be broken or encrypted: {pdf_path.name}")
from exc

Explanation: Raises an error so the caller knows processing cannot continue normally.

Line 356: except Exception as exc:

Explanation: Handles an error raised inside the matching try block.

Line 357: raise RuntimeError(f"Could not inspect PDF page count for {pdf_path.name}: {exc}") from exc

Explanation: Raises an error so the caller knows processing cannot continue normally.

Line 358: [blank]

Explanation: Blank line used to separate logical sections.

Line 359: [blank]

Explanation: Blank line used to separate logical sections.

Line 360: def convert_pdf_page_to_image(pdf_path: Path, config: Config, page_number: int) -> Image.Image:

Explanation: Converts one PDF page into a PIL image.

Line 361: kwargs = pdf2image_kwargs(config)

Explanation: Assigns or updates the value of kwargs.

Line 362: kwargs["first_page"] = page_number

Explanation: Assigns or updates the value of kwargs["first_page"].

Line 363: kwargs["last_page"] = page_number

Explanation: Assigns or updates the value of kwargs["last_page"].

Line 364: try:

Explanation: Starts a protected block where errors can be caught.

Line 365: images = convert_from_path(str(pdf_path), **kwargs)

Explanation: Converts PDF pages into images using Poppler/pdf2image.

Line 366: if not images:

Explanation: Starts a conditional branch; the indented block runs only when this condition is true.

Line 367: raise RuntimeError(f"No image was produced for page {page_number}")

Explanation: Raises an error so the caller knows processing cannot continue normally.

Line 368: return images[0]

Explanation: Returns the result from convert_pdf_page_to_image.

Line 369: except PDFInfoNotInstalledError as exc:

Explanation: Handles an error raised inside the matching try block.

Line 370: message = (

Explanation: Assigns or updates the value of message.

Line 371: "Poppler is not installed or POPPLER_PATH is wrong. On Windows, install Poppler and set "

Explanation: Continues the logic inside convert_pdf_page_to_image.

Line 372: "POPPLER_PATH=C:\\poppler\\bin in .env, then run again."

Explanation: Continues the logic inside convert_pdf_page_to_image.

Line 373:)

Explanation: Closes a multi-line expression or data structure.

Line 374: raise RuntimeError(message) from exc

Explanation: Raises an error so the caller knows processing cannot continue normally.

Line 375: except (PDFPageCountError, PDFSyntaxError) as exc:

Explanation: Handles an error raised inside the matching try block.

Line 376: raise RuntimeError(f"Could not read PDF pages. The file may be broken or encrypted: {pdf_path.name}")
from exc

Explanation: Raises an error so the caller knows processing cannot continue normally.

Line 377: except Exception as exc:

Explanation: Handles an error raised inside the matching try block.

Line 378: raise RuntimeError(f"PDF page conversion failed for {pdf_path.name} page {page_number}: {exc}") from exc

Explanation: Raises an error so the caller knows processing cannot continue normally.

Line 379: [blank]

Explanation: Blank line used to separate logical sections.

Line 380: [blank]

Explanation: Blank line used to separate logical sections.

Line 381: def pil_image_to_bytes(image: Image.Image) -> bytes:

Explanation: Converts a PIL image into PNG bytes for Gemini.

Line 382: buffer = io.BytesIO()

Explanation: Assigns or updates the value of buffer.

Line 383: if image.mode not in {"RGB", "L"}:

Explanation: Starts a conditional branch; the indented block runs only when this condition is true.

Line 384: image = image.convert("RGB")

Explanation: Assigns or updates the value of image.

Line 385: image.save(buffer, format="PNG")

Explanation: Assigns or updates the value of image.save(buffer, format).

Line 386: return buffer.getvalue()

Explanation: Returns the result from pil_image_to_bytes.

Line 387: [blank]

Explanation: Blank line used to separate logical sections.

Line 388: [blank]

Explanation: Blank line used to separate logical sections.

Line 389: def is_retryable_error(error: Exception) -> bool:

Explanation: Detects Gemini/network errors that should be retried.

Line 390: text = str(error).lower()

Explanation: Assigns or updates the value of text.

Line 391: `retry_terms = [`
Explanation: Assigns or updates the value of `retry_terms`.

Line 392: `"429",`
Explanation: Continues the logic inside `is_retryable_error`.

Line 393: `"quota",`
Explanation: Continues the logic inside `is_retryable_error`.

Line 394: `"rate limit",`
Explanation: Continues the logic inside `is_retryable_error`.

Line 395: `"resource exhausted",`
Explanation: Continues the logic inside `is_retryable_error`.

Line 396: `"too many requests",`
Explanation: Continues the logic inside `is_retryable_error`.

Line 397: `"503",`
Explanation: Continues the logic inside `is_retryable_error`.

Line 398: `"unavailable",`
Explanation: Continues the logic inside `is_retryable_error`.

Line 399: `"deadline",`
Explanation: Continues the logic inside `is_retryable_error`.

Line 400: `"timeout",`
Explanation: Continues the logic inside `is_retryable_error`.

Line 401: `"temporarily",`
Explanation: Continues the logic inside `is_retryable_error`.

Line 402: `"connection",`
Explanation: Continues the logic inside `is_retryable_error`.

Line 403: `"network",`
Explanation: Continues the logic inside `is_retryable_error`.

Line 404: `]`
Explanation: Continues the logic inside `is_retryable_error`.

Line 405: `return any(term in text for term in retry_terms)`
Explanation: Returns the result from `is_retryable_error`.

Line 406: [blank]
Explanation: Blank line used to separate logical sections.

Line 407: [blank]
Explanation: Blank line used to separate logical sections.

Line 408: `def response_text(response: Any) -> str:`
Explanation: Extracts text from a Gemini SDK response.

Line 409: `text = getattr(response, "text", None)`
Explanation: Assigns or updates the value of `text`.

Line 410: `if text:`
Explanation: Starts a conditional branch; the indented block runs only when this condition is true.

Line 411: `return str(text).strip()`
Explanation: Returns the result from `response_text`.

Line 412: `try:`
Explanation: Starts a protected block where errors can be caught.

Line 413: `candidates = getattr(response, "candidates", None) or []`
Explanation: Assigns or updates the value of `candidates`.

Line 414: `parts: list[str] = []`
Explanation: Assigns or updates the value of `parts: list[str]`.

Line 415: `for candidate in candidates:`
Explanation: Starts a loop over each item in a collection.

Line 416: content = getattr(candidate, "content", None)
Explanation: Assigns or updates the value of content.

Line 417: for part in getattr(content, "parts", []) or []:
Explanation: Starts a loop over each item in a collection.

Line 418: part_text = getattr(part, "text", None)
Explanation: Assigns or updates the value of part_text.

Line 419: if part_text:
Explanation: Starts a conditional branch; the indented block runs only when this condition is true.

Line 420: parts.append(str(part_text))
Explanation: Continues the logic inside response_text.

Line 421: return "\n".join(parts).strip()
Explanation: Returns the result from response_text.

Line 422: except Exception:
Explanation: Handles an error raised inside the matching try block.

Line 423: return ""
Explanation: Returns the result from response_text.

Line 424: [blank]
Explanation: Blank line used to separate logical sections.

Line 425: [blank]
Explanation: Blank line used to separate logical sections.

Line 426: def gemini_client(config: Config) -> genai.Client:
Explanation: Creates a Gemini client with the configured timeout.

Line 427: return genai.Client(
Explanation: Returns the result from gemini_client.

Line 428: api_key=config.gemini_api_key,
Explanation: Assigns or updates the value of api_key.

Line 429: http_options=types.HttpOptions(timeout=config.gemini_timeout_seconds * 1000),
Explanation: Assigns or updates the value of http_options.

Line 430:)
Explanation: Closes a multi-line expression or data structure.

Line 431: [blank]
Explanation: Blank line used to separate logical sections.

Line 432: [blank]
Explanation: Blank line used to separate logical sections.

Line 433: def call_gemini_with_retry(
Explanation: Calls Gemini with retries, delay, timeout, and state tracking.

Line 434: client: genai.Client,
Explanation: Creates a Gemini SDK client for API calls.

Line 435: config: Config,
Explanation: Continues the logic inside call_gemini_with_retry.

Line 436: state: dict[str, Any],
Explanation: Continues the logic inside call_gemini_with_retry.

Line 437: retry_key: str,
Explanation: Continues the logic inside call_gemini_with_retry.

Line 438: contents: list[Any],
Explanation: Continues the logic inside call_gemini_with_retry.

Line 439:) -> str:
Explanation: Continues the logic inside call_gemini_with_retry.

Line 440: last_error: Exception | None = None
Explanation: Continues the logic inside call_gemini_with_retry.

Line 441: for attempt in range(1, config.max_retries + 1):
Explanation: Starts a loop over each item in a collection.

Line 442: if config.gemini_delay_seconds:
Explanation: Starts a conditional branch; the indented block runs only when this condition is true.

Line 443: time.sleep(config.gemini_delay_seconds)
Explanation: Continues the logic inside call_gemini_with_retry.

Line 444: try:
Explanation: Starts a protected block where errors can be caught.

Line 445: response = client.models.generate_content(model=config.gemini_model, contents=contents)
Explanation: Sends content to Gemini and waits for the model response.

Line 446: text = response_text(response)
Explanation: Assigns or updates the value of text.

Line 447: if not text:
Explanation: Starts a conditional branch; the indented block runs only when this condition is true.

Line 448: raise RuntimeError("Empty Gemini response")
Explanation: Raises an error so the caller knows processing cannot continue normally.

Line 449: return text
Explanation: Returns the result from call_gemini_with_retry.

Line 450: except Exception as exc:
Explanation: Handles an error raised inside the matching try block.

Line 451: last_error = exc
Explanation: Assigns or updates the value of last_error.

Line 452: state.setdefault("retry_counts", {})
Explanation: Continues the logic inside call_gemini_with_retry.

Line 453: state["retry_counts"][retry_key] = int(state["retry_counts"].get(retry_key, 0)) + 1
Explanation: Assigns or updates the value of state["retry_counts"][retry_key].

Line 454: save_state(config, state)
Explanation: Continues the logic inside call_gemini_with_retry.

Line 455: retryable = is_retryable_error(exc)
Explanation: Assigns or updates the value of retryable.

Line 456: logging.warning(
Explanation: Writes diagnostic information into the processor log.

Line 457: "Gemini call failed for %s on attempt %s/%s: %s",
Explanation: Continues the logic inside call_gemini_with_retry.

Line 458: retry_key,
Explanation: Continues the logic inside call_gemini_with_retry.

Line 459: attempt,
Explanation: Continues the logic inside call_gemini_with_retry.

Line 460: config.max_retries,
Explanation: Continues the logic inside call_gemini_with_retry.

Line 461: exc,
Explanation: Continues the logic inside call_gemini_with_retry.

Line 462:)
Explanation: Closes a multi-line expression or data structure.

Line 463: if attempt >= config.max_retries:
Explanation: Starts a conditional branch; the indented block runs only when this condition is true.

Line 464: break
Explanation: Continues the logic inside call_gemini_with_retry.

Line 465: sleep_seconds = 60 if retryable else min(60, 2**attempt)
Explanation: Assigns or updates the value of sleep_seconds.

Line 466: if retryable:

Explanation: Starts a conditional branch; the indented block runs only when this condition is true.

Line 467: print(f"Gemini is rate-limited or unavailable for {retry_key}. Sleeping {sleep_seconds} seconds before retry.")

Explanation: Writes progress text to the console or job log.

Line 468: time.sleep(sleep_seconds)

Explanation: Continues the logic inside call_gemini_with_retry.

Line 469: raise RuntimeError(f"Gemini failed after {config.max_retries} attempts for {retry_key}: {last_error}")

Explanation: Raises an error so the caller knows processing cannot continue normally.

Line 470: [blank]

Explanation: Blank line used to separate logical sections.

Line 471: [blank]

Explanation: Blank line used to separate logical sections.

Line 472: def image_part(image_bytes: bytes) -> types.Part:

Explanation: Wraps PNG bytes as a Gemini image part.

Line 473: return types.Part.from_bytes(data=image_bytes, mime_type="image/png")

Explanation: Returns the result from image_part.

Line 474: [blank]

Explanation: Blank line used to separate logical sections.

Line 475: [blank]

Explanation: Blank line used to separate logical sections.

Line 476: def extract_page_text(

Explanation: Sends a page image to Gemini for raw transcription.

Line 477: client: genai.Client,

Explanation: Creates a Gemini SDK client for API calls.

Line 478: config: Config,

Explanation: Continues the logic inside extract_page_text.

Line 479: state: dict[str, Any],

Explanation: Continues the logic inside extract_page_text.

Line 480: pdf_stem: str,

Explanation: Continues the logic inside extract_page_text.

Line 481: page_number: int,

Explanation: Continues the logic inside extract_page_text.

Line 482: image: Image.Image,

Explanation: Continues the logic inside extract_page_text.

Line 483:) -> str:

Explanation: Continues the logic inside extract_page_text.

Line 484: image_bytes = pil_image_to_bytes(image)

Explanation: Assigns or updates the value of image_bytes.

Line 485: retry_key = f"{pdf_stem}:page_{page_number:03d}:extract"

Explanation: Assigns or updates the value of retry_key.

Line 486: return call_gemini_with_retry(

Explanation: Returns the result from extract_page_text.

Line 487: client,

Explanation: Continues the logic inside extract_page_text.

Line 488: config,

Explanation: Continues the logic inside extract_page_text.

Line 489: state,

Explanation: Continues the logic inside extract_page_text.

Line 490: retry_key,
Explanation: Continues the logic inside extract_page_text.

Line 491: [EXTRACTION_PROMPT, image_part(image_bytes)],
Explanation: Starts or continues a multi-line expression or data structure.

Line 492:)
Explanation: Closes a multi-line expression or data structure.

Line 493: [blank]
Explanation: Blank line used to separate logical sections.

Line 494: [blank]
Explanation: Blank line used to separate logical sections.

Line 495: def verify_page_text(
Explanation: Sends image and raw text to Gemini for verification.

Line 496: client: genai.Client,
Explanation: Creates a Gemini SDK client for API calls.

Line 497: config: Config,
Explanation: Continues the logic inside verify_page_text.

Line 498: state: dict[str, Any],
Explanation: Continues the logic inside verify_page_text.

Line 499: pdf_stem: str,
Explanation: Continues the logic inside verify_page_text.

Line 500: page_number: int,
Explanation: Continues the logic inside verify_page_text.

Line 501: image: Image.Image,
Explanation: Continues the logic inside verify_page_text.

Line 502: extracted_text: str,
Explanation: Continues the logic inside verify_page_text.

Line 503:) -> str:
Explanation: Continues the logic inside verify_page_text.

Line 504: image_bytes = pil_image_to_bytes(image)
Explanation: Assigns or updates the value of image_bytes.

Line 505: retry_key = f"{pdf_stem}:page_{page_number:03d}:verify"
Explanation: Assigns or updates the value of retry_key.

Line 506: prompt = f"{VERIFICATION_PROMPT}\n\nExtracted text to verify:\n\n{extracted_text}"
Explanation: Assigns or updates the value of prompt.

Line 507: return call_gemini_with_retry(
Explanation: Returns the result from verify_page_text.

Line 508: client,
Explanation: Continues the logic inside verify_page_text.

Line 509: config,
Explanation: Continues the logic inside verify_page_text.

Line 510: state,
Explanation: Continues the logic inside verify_page_text.

Line 511: retry_key,
Explanation: Continues the logic inside verify_page_text.

Line 512: [prompt, image_part(image_bytes)],
Explanation: Starts or continues a multi-line expression or data structure.

Line 513:)
Explanation: Closes a multi-line expression or data structure.

Line 514: [blank]
Explanation: Blank line used to separate logical sections.

Line 515: [blank]

Explanation: Blank line used to separate logical sections.

Line 516: `def page_file(directory: Path, page_number: int, suffix: str) -> Path:`

Explanation: Builds a standard per-page filename.

Line 517: `return directory / f"page_{page_number:03d}_{suffix}.md"`

Explanation: Returns the result from `page_file`.

Line 518: [blank]

Explanation: Blank line used to separate logical sections.

Line 519: [blank]

Explanation: Blank line used to separate logical sections.

Line 520: `def int_list(state: dict[str, Any], section: str, pdf_stem: str) -> list[int]:`

Explanation: Reads a list of page numbers from processing state.

Line 521: `values = state.setdefault(section, {}).setdefault(pdf_stem, [])`

Explanation: Assigns or updates the value of `values`.

Line 522: `cleaned: list[int] = []`

Explanation: Assigns or updates the value of `cleaned`: `list[int]`.

Line 523: `for value in values:`

Explanation: Starts a loop over each item in a collection.

Line 524: `try:`

Explanation: Starts a protected block where errors can be caught.

Line 525: `cleaned.append(int(value))`

Explanation: Continues the logic inside `int_list`.

Line 526: `except (TypeError, ValueError):`

Explanation: Handles an error raised inside the matching try block.

Line 527: `continue`

Explanation: Continues the logic inside `int_list`.

Line 528: `return sorted(set(cleaned))`

Explanation: Returns the result from `int_list`.

Line 529: [blank]

Explanation: Blank line used to separate logical sections.

Line 530: [blank]

Explanation: Blank line used to separate logical sections.

Line 531: `def add_page_state(config: Config, state: dict[str, Any], section: str, pdf_stem: str, page_number: int) -> None:`

Explanation: Adds one page number to a state section.

Line 532: `pages = set(int_list(state, section, pdf_stem))`

Explanation: Assigns or updates the value of `pages`.

Line 533: `pages.add(page_number)`

Explanation: Continues the logic inside `add_page_state`.

Line 534: `state.setdefault(section, {})[pdf_stem] = sorted(pages)`

Explanation: Assigns or updates the value of `state.setdefault(section, {})[pdf_stem]`.

Line 535: `save_state(config, state)`

Explanation: Continues the logic inside `add_page_state`.

Line 536: [blank]

Explanation: Blank line used to separate logical sections.

Line 537: [blank]

Explanation: Blank line used to separate logical sections.

Line 538: `def mark_page_extracted(config: Config, state: dict[str, Any], pdf_stem: str, page_number: int) -> None:`

Explanation: Marks a page as raw-extracted.

Line 539: `add_page_state(config, state, "extracted_pages", pdf_stem, page_number)`

Explanation: Continues the logic inside `mark_page_extracted`.

Line 540: [blank]

Explanation: Blank line used to separate logical sections.

Line 541: [blank]

Explanation: Blank line used to separate logical sections.

Line 542: `def mark_page_verified(config: Config, state: dict[str, Any], pdf_stem: str, page_number: int) -> None:`

Explanation: Marks a page verified and removes any old failure marker.

Line 543: `add_page_state(config, state, "verified_pages", pdf_stem, page_number)`

Explanation: Continues the logic inside mark_page_verified.

Line 544: `add_page_state(config, state, "completed_pages", pdf_stem, page_number)`

Explanation: Continues the logic inside mark_page_verified.

Line 545: `failed = state.setdefault("failed_pages", {}).setdefault(pdf_stem, {})`

Explanation: Assigns or updates the value of failed.

Line 546: `failed.pop(str(page_number), None)`

Explanation: Continues the logic inside mark_page_verified.

Line 547: `if not failed:`

Explanation: Starts a conditional branch; the indented block runs only when this condition is true.

Line 548: `state.get("failed_pages", {}).pop(pdf_stem, None)`

Explanation: Continues the logic inside mark_page_verified.

Line 549: `save_state(config, state)`

Explanation: Continues the logic inside mark_page_verified.

Line 550: [blank]

Explanation: Blank line used to separate logical sections.

Line 551: [blank]

Explanation: Blank line used to separate logical sections.

Line 552: `def mark_page_failed(`

Explanation: Records a page-level failure.

Line 553: `config: Config,`

Explanation: Continues the logic inside mark_page_failed.

Line 554: `state: dict[str, Any],`

Explanation: Continues the logic inside mark_page_failed.

Line 555: `pdf_stem: str,`

Explanation: Continues the logic inside mark_page_failed.

Line 556: `page_number: int,`

Explanation: Continues the logic inside mark_page_failed.

Line 557: `stage: str,`

Explanation: Continues the logic inside mark_page_failed.

Line 558: `error: Exception,`

Explanation: Continues the logic inside mark_page_failed.

Line 559: `) -> None:`

Explanation: Continues the logic inside mark_page_failed.

Line 560: `failed = state.setdefault("failed_pages", {}).setdefault(pdf_stem, {})`

Explanation: Assigns or updates the value of failed.

Line 561: `failed[str(page_number)] = {"stage": stage, "error": str(error), "timestamp": utc_now()}`

Explanation: Assigns or updates the value of failed[str(page_number)].

Line 562: `save_state(config, state)`

Explanation: Continues the logic inside mark_page_failed.

Line 563: [blank]

Explanation: Blank line used to separate logical sections.

Line 564: [blank]

Explanation: Blank line used to separate logical sections.

Line 565: `def mark_pdf_complete(config: Config, state: dict[str, Any], pdf_stem: str) -> None:`

Explanation: Marks a full PDF as complete.

Line 566: `completed = set(state.setdefault("completed_pdfs", []))`

Explanation: Assigns or updates the value of completed.

Line 567: `completed.add(pdf_stem)`

Explanation: Continues the logic inside mark_pdf_complete.

Line 568: `state["completed_pdfs"] = sorted(completed)`

Explanation: Assigns or updates the value of state["completed_pdfs"].

Line 569: `save_state(config, state)`

Explanation: Continues the logic inside mark_pdf_complete.

Line 570: [blank]

Explanation: Blank line used to separate logical sections.

Line 571: [blank]

Explanation: Blank line used to separate logical sections.

Line 572: `def combine_verified_pages(`

Explanation: Combines verified per-page text files into one chapter source file.

Line 573: `config: Config,`

Explanation: Continues the logic inside combine_verified_pages.

Line 574: `pdf_stem: str,`

Explanation: Continues the logic inside combine_verified_pages.

Line 575: `page_numbers: list[int],`

Explanation: Continues the logic inside combine_verified_pages.

Line 576: `output_stem: str | None = None,`

Explanation: Continues the logic inside combine_verified_pages.

Line 577: `) -> Path:`

Explanation: Continues the logic inside combine_verified_pages.

Line 578: `source_name = output_stem or pdf_stem`

Explanation: Assigns or updates the value of source_name.

Line 579: `source_path = config.chapter_sources_dir / f"{source_name}_verified_source.md"`

Explanation: Assigns or updates the value of source_path.

Line 580: `verified_pdf_dir = config.verified_dir / pdf_stem`

Explanation: Assigns or updates the value of verified_pdf_dir.

Line 581: `pieces: list[str] = []`

Explanation: Assigns or updates the value of pieces: list[str].

Line 582: `for page_number in page_numbers:`

Explanation: Starts a loop over each item in a collection.

Line 583: `verified_path = page_file(verified_pdf_dir, page_number, "verified")`

Explanation: Assigns or updates the value of verified_path.

Line 584: `if not verified_path.exists():`

Explanation: Starts a conditional branch; the indented block runs only when this condition is true.

Line 585: `raise RuntimeError(f"Missing verified page file: {verified_path}")`

Explanation: Raises an error so the caller knows processing cannot continue normally.

Line 586: `text = verified_path.read_text(encoding="utf-8").strip()`

Explanation: Reads text content from a file.

Line 587: `pieces.append(f"<!-- PAGE {page_number} START -->\n{text}\n<!-- PAGE {page_number} END -->")`

Explanation: Continues the logic inside combine_verified_pages.

Line 588: `source_path.write_text("\n\n".join(pieces).strip() + "\n", encoding="utf-8")`

Explanation: Writes text content to a file.

Line 589: `return source_path`

Explanation: Returns the result from combine_verified_pages.

Line 590: [blank]

Explanation: Blank line used to separate logical sections.

Line 591: [blank]

Explanation: Blank line used to separate logical sections.

Line 592: `def format_chapter_notes(`

Explanation: Creates structured study notes from verified source text.

Line 593: `client: genai.Client,`

Explanation: Creates a Gemini SDK client for API calls.

Line 594: `config: Config,`

Explanation: Continues the logic inside `format_chapter_notes`.

Line 595: `state: dict[str, Any],`

Explanation: Continues the logic inside `format_chapter_notes`.

Line 596: `pdf_path: Path,`

Explanation: Continues the logic inside `format_chapter_notes`.

Line 597: `source_path: Path,`

Explanation: Continues the logic inside `format_chapter_notes`.

Line 598: `page_range_text: str,`

Explanation: Continues the logic inside `format_chapter_notes`.

Line 599: `) -> str:`

Explanation: Continues the logic inside `format_chapter_notes`.

Line 600: `source_text = source_path.read_text(encoding="utf-8")`

Explanation: Reads text content from a file.

Line 601: `prompt = (`

Explanation: Assigns or updates the value of `prompt`.

Line 602: `f"{FORMAT_PROMPT}\n\n"`

Explanation: Continues the logic inside `format_chapter_notes`.

Line 603: `f"Source PDF filename: {pdf_path.name}\n"`

Explanation: Continues the logic inside `format_chapter_notes`.

Line 604: `f"Verified page range: {page_range_text}\n\n"`

Explanation: Continues the logic inside `format_chapter_notes`.

Line 605: `f"Verified source text:\n\n{source_text}"`

Explanation: Continues the logic inside `format_chapter_notes`.

Line 606: `)`

Explanation: Closes a multi-line expression or data structure.

Line 607: `return call_gemini_with_retry(`

Explanation: Returns the result from `format_chapter_notes`.

Line 608: `client,`

Explanation: Continues the logic inside `format_chapter_notes`.

Line 609: `config,`

Explanation: Continues the logic inside `format_chapter_notes`.

Line 610: `state,`

Explanation: Continues the logic inside `format_chapter_notes`.

Line 611: `f"{pdf_path.stem}:format_chapter",`

Explanation: Continues the logic inside `format_chapter_notes`.

Line 612: `[prompt],`

Explanation: Starts or continues a multi-line expression or data structure.

Line 613: `)`

Explanation: Closes a multi-line expression or data structure.

Line 614: [blank]

Explanation: Blank line used to separate logical sections.

Line 615: [blank]

Explanation: Blank line used to separate logical sections.

Line 616: `def chapter_already_in_master(config: Config, pdf_name: str) -> bool:`

Explanation: Checks if a PDF has already been appended to master notes.

Line 617: `if not config.master_notes_file.exists():`

Explanation: Starts a conditional branch; the indented block runs only when this condition is true.

Line 618: `return False`

Explanation: Returns the result from `chapter_already_in_master`.

Line 619: `marker = f"# END OF CHAPTER: {pdf_name}"`

Explanation: Assigns or updates the value of `marker`.

Line 620: `try:`

Explanation: Starts a protected block where errors can be caught.

Line 621: `return marker in config.master_notes_file.read_text(encoding="utf-8")`

Explanation: Returns the result from `chapter_already_in_master`.

Line 622: `except Exception:`

Explanation: Handles an error raised inside the matching try block.

Line 623: `logging.warning("Could not inspect master notes file. State file will be used for append decision.")`

Explanation: Writes diagnostic information into the processor log.

Line 624: `return False`

Explanation: Returns the result from `chapter_already_in_master`.

Line 625: [blank]

Explanation: Blank line used to separate logical sections.

Line 626: [blank]

Explanation: Blank line used to separate logical sections.

Line 627: `def append_to_master_notes(config: Config, state: dict[str, Any], pdf_name: str, notes: str) -> None:`

Explanation: Appends final notes to `Complete_Book_Notes.md` once.

Line 628: `appended = set(state.setdefault("master_appended_chapters", []))`

Explanation: Assigns or updates the value of `appended`.

Line 629: `if pdf_name in appended or chapter_already_in_master(config, pdf_name):`

Explanation: Starts a conditional branch; the indented block runs only when this condition is true.

Line 630: `logging.info("Master notes already contain %s. Skipping append.", pdf_name)`

Explanation: Writes diagnostic information into the processor log.

Line 631: `return`

Explanation: Continues the logic inside `append_to_master_notes`.

Line 632: [blank]

Explanation: Blank line used to separate logical sections.

Line 633: `with config.master_notes_file.open("a", encoding="utf-8") as handle:`

Explanation: Opens a context-managed resource and closes it automatically afterward.

Line 634: `if config.master_notes_file.stat().st_size:`

Explanation: Starts a conditional branch; the indented block runs only when this condition is true.

Line 635: `handle.write("\n\n")`

Explanation: Continues the logic inside `append_to_master_notes`.

Line 636: `handle.write(notes.strip())`

Explanation: Continues the logic inside `append_to_master_notes`.

Line 637: `handle.write(f"\n\n---\n\n# END OF CHAPTER: {pdf_name}\n\n---\n\n")`

Explanation: Continues the logic inside `append_to_master_notes`.

Line 638: `appended.add(pdf_name)`

Explanation: Continues the logic inside `append_to_master_notes`.

Line 639: `state["master_appended_chapters"] = sorted(appended)`

Explanation: Assigns or updates the value of `state["master_appended_chapters"]`.

Line 640: save_state(config, state)

Explanation: Continues the logic inside append_to_master_notes.

Line 641: [blank]

Explanation: Blank line used to separate logical sections.

Line 642: [blank]

Explanation: Blank line used to separate logical sections.

Line 643: def create_docx_optional(

Explanation: Creates a DOCX from exact verified page text.

Line 644: config: Config,

Explanation: Continues the logic inside create_docx_optional.

Line 645: pdf_stem: str,

Explanation: Continues the logic inside create_docx_optional.

Line 646: page_numbers: list[int],

Explanation: Continues the logic inside create_docx_optional.

Line 647: output_stem: str | None = None,

Explanation: Continues the logic inside create_docx_optional.

Line 648:) -> Path | None:

Explanation: Continues the logic inside create_docx_optional.

Line 649: if not config.create_docx:

Explanation: Starts a conditional branch; the indented block runs only when this condition is true.

Line 650: return None

Explanation: Returns the result from create_docx_optional.

Line 651: try:

Explanation: Starts a protected block where errors can be caught.

Line 652: from docx import Document

Explanation: Imports specific classes or functions from another module.

Line 653: except Exception as exc:

Explanation: Handles an error raised inside the matching try block.

Line 654: logging.error("CREATE_DOCX=true, but python-docx is unavailable: %s", exc)

Explanation: Writes diagnostic information into the processor log.

Line 655: return None

Explanation: Returns the result from create_docx_optional.

Line 656: [blank]

Explanation: Blank line used to separate logical sections.

Line 657: docx_stem = output_stem or f"{pdf_stem}_verified"

Explanation: Assigns or updates the value of docx_stem.

Line 658: document = Document()

Explanation: Assigns or updates the value of document.

Line 659: document.add_heading(pdf_stem, level=1)

Explanation: Assigns or updates the value of document.add_heading(pdf_stem, level.

Line 660: verified_pdf_dir = config.verified_dir / pdf_stem

Explanation: Assigns or updates the value of verified_pdf_dir.

Line 661: for page_number in page_numbers:

Explanation: Starts a loop over each item in a collection.

Line 662: verified_path = page_file(verified_pdf_dir, page_number, "verified")

Explanation: Assigns or updates the value of verified_path.

Line 663: if not verified_path.exists():

Explanation: Starts a conditional branch; the indented block runs only when this condition is true.

Line 664: continue

Explanation: Continues the logic inside create_docx_optional.

Line 665: document.add_heading(f"Page {page_number}", level=2)
Explanation: Assigns or updates the value of document.add_heading(f"Page {page_number}", level.

Line 666: text = verified_path.read_text(encoding="utf-8").strip()
Explanation: Reads text content from a file.

Line 667: for paragraph in text.splitlines():
Explanation: Starts a loop over each item in a collection.

Line 668: document.add_paragraph(paragraph)
Explanation: Continues the logic inside create_docx_optional.

Line 669: output_path = config.output_dir / f"{docx_stem}.docx"
Explanation: Assigns or updates the value of output_path.

Line 670: document.save(output_path)
Explanation: Continues the logic inside create_docx_optional.

Line 671: logging.info("DOCX created from verified text: %s", output_path)
Explanation: Writes diagnostic information into the processor log.

Line 672: return output_path
Explanation: Returns the result from create_docx_optional.

Line 673: [blank]
Explanation: Blank line used to separate logical sections.

Line 674: [blank]
Explanation: Blank line used to separate logical sections.

Line 675: def add_markdown_line_to_docx(document: Any, line: str) -> None:
Explanation: Converts a Markdown-like line into a Word paragraph.

Line 676: stripped = line.strip()
Explanation: Assigns or updates the value of stripped.

Line 677: if not stripped:
Explanation: Starts a conditional branch; the indented block runs only when this condition is true.

Line 678: return
Explanation: Continues the logic inside add_markdown_line_to_docx.

Line 679: [blank]
Explanation: Blank line used to separate logical sections.

Line 680: if stripped.startswith("# "):
Explanation: Starts a conditional branch; the indented block runs only when this condition is true.

Line 681: document.add_heading(stripped[2:].strip(), level=1)
Explanation: Assigns or updates the value of document.add_heading(stripped[2:].strip(), level.

Line 682: elif stripped.startswith("## "):
Explanation: Starts another conditional branch if the previous conditions were false.

Line 683: document.add_heading(stripped[3:].strip(), level=2)
Explanation: Assigns or updates the value of document.add_heading(stripped[3:].strip(), level.

Line 684: elif stripped.startswith("### "):
Explanation: Starts another conditional branch if the previous conditions were false.

Line 685: document.add_heading(stripped[4:].strip(), level=3)
Explanation: Assigns or updates the value of document.add_heading(stripped[4:].strip(), level.

Line 686: elif stripped.startswith("#### "):
Explanation: Starts another conditional branch if the previous conditions were false.

Line 687: document.add_heading(stripped[5:].strip(), level=4)
Explanation: Assigns or updates the value of document.add_heading(stripped[5:].strip(), level.

Line 688: elif stripped.startswith("- "):
Explanation: Starts another conditional branch if the previous conditions were false.

Line 689: document.add_paragraph(stripped[2:].strip(), style="List Bullet")
Explanation: Assigns or updates the value of document.add_paragraph(stripped[2:].strip(), style.

Line 690: `elif stripped == "---":`

Explanation: Starts another conditional branch if the previous conditions were false.

Line 691: `document.add_paragraph("")`

Explanation: Continues the logic inside `add_markdown_line_to_docx`.

Line 692: `else:`

Explanation: Runs the indented block when previous conditions did not match.

Line 693: `document.add_paragraph(stripped)`

Explanation: Continues the logic inside `add_markdown_line_to_docx`.

Line 694: [blank]

Explanation: Blank line used to separate logical sections.

Line 695: [blank]

Explanation: Blank line used to separate logical sections.

Line 696: `def create_formatted_notes_docx(config: Config, notes_path: Path) -> Path | None:`

Explanation: Creates a DOCX from structured notes.

Line 697: `if not config.create_docx:`

Explanation: Starts a conditional branch; the indented block runs only when this condition is true.

Line 698: `return None`

Explanation: Returns the result from `create_formatted_notes_docx`.

Line 699: `try:`

Explanation: Starts a protected block where errors can be caught.

Line 700: `from docx import Document`

Explanation: Imports specific classes or functions from another module.

Line 701: `except Exception as exc:`

Explanation: Handles an error raised inside the matching try block.

Line 702: `logging.error("CREATE_DOCX=true, but python-docx is unavailable: %s", exc)`

Explanation: Writes diagnostic information into the processor log.

Line 703: `return None`

Explanation: Returns the result from `create_formatted_notes_docx`.

Line 704: [blank]

Explanation: Blank line used to separate logical sections.

Line 705: `docx_path = notes_path.with_suffix(".docx")`

Explanation: Assigns or updates the value of `docx_path`.

Line 706: `document = Document()`

Explanation: Assigns or updates the value of `document`.

Line 707: `for line in notes_path.read_text(encoding="utf-8").splitlines():`

Explanation: Starts a loop over each item in a collection.

Line 708: `add_markdown_line_to_docx(document, line)`

Explanation: Continues the logic inside `create_formatted_notes_docx`.

Line 709: `document.save(docx_path)`

Explanation: Continues the logic inside `create_formatted_notes_docx`.

Line 710: `logging.info("DOCX created from formatted notes: %s", docx_path)`

Explanation: Writes diagnostic information into the processor log.

Line 711: `return docx_path`

Explanation: Returns the result from `create_formatted_notes_docx`.

Line 712: [blank]

Explanation: Blank line used to separate logical sections.

Line 713: [blank]

Explanation: Blank line used to separate logical sections.

Line 714: `def is_chapter_fully_verified(state: dict[str, Any], pdf_stem: str, page_numbers: list[int]) -> bool:`

Explanation: Checks whether every expected page is verified.

Line 715: verified = set(int_list(state, "verified_pages", pdf_stem))

Explanation: Assigns or updates the value of verified.

Line 716: return all(page_number in verified for page_number in page_numbers)

Explanation: Returns the result from is_chapter_fully_verified.

Line 717: [blank]

Explanation: Blank line used to separate logical sections.

Line 718: [blank]

Explanation: Blank line used to separate logical sections.

Line 719: def process_pdf(client: genai.Client, config: Config, state: dict[str, Any], pdf_path: Path) -> None:

Explanation: Runs the full processing workflow for one PDF.

Line 720: pdf_stem = pdf_path.stem

Explanation: Assigns or updates the value of pdf_stem.

Line 721: if not config.test_max_pages and pdf_stem in set(state.get("completed_pdfs", [])):

Explanation: Starts a conditional branch; the indented block runs only when this condition is true.

Line 722: logging.info("Skipping completed PDF before conversion: %s", pdf_path.name)

Explanation: Writes diagnostic information into the processor log.

Line 723: print(f"Skipping completed PDF: {pdf_path.name}")

Explanation: Writes progress text to the console or job log.

Line 724: return

Explanation: Continues the logic inside process_pdf.

Line 725: [blank]

Explanation: Blank line used to separate logical sections.

Line 726: logging.info("Starting PDF: %s", pdf_path.name)

Explanation: Writes diagnostic information into the processor log.

Line 727: print(f"\nProcessing PDF: {pdf_path.name}")

Explanation: Writes progress text to the console or job log.

Line 728: [blank]

Explanation: Blank line used to separate logical sections.

Line 729: try:

Explanation: Starts a protected block where errors can be caught.

Line 730: total_pages = get_pdf_page_count(pdf_path, config)

Explanation: Assigns or updates the value of total_pages.

Line 731: except Exception as exc:

Explanation: Handles an error raised inside the matching try block.

Line 732: logging.error("%s", exc)

Explanation: Writes diagnostic information into the processor log.

Line 733: state.setdefault("failed_pages", {}).setdefault(pdf_stem, {})[PDF] = {

Explanation: Assigns or updates the value of state.setdefault("failed_pages", {}).setdefault(pdf_stem, {})[PDF].

Line 734: "stage": "get_pdf_page_count",

Explanation: Continues the logic inside process_pdf.

Line 735: "error": str(exc),

Explanation: Continues the logic inside process_pdf.

Line 736: "timestamp": utc_now(),

Explanation: Continues the logic inside process_pdf.

Line 737: }

Explanation: Closes a multi-line expression or data structure.

Line 738: save_state(config, state)

Explanation: Continues the logic inside process_pdf.

Line 739: return

Explanation: Continues the logic inside process_pdf.

Line 740: [blank]

Explanation: Blank line used to separate logical sections.

Line 741: if config.test_max_pages:

Explanation: Starts a conditional branch; the indented block runs only when this condition is true.

Line 742: start_page = min(config.test_start_page, total_pages)

Explanation: Assigns or updates the value of start_page.

Line 743: end_page = min(total_pages, start_page + config.test_max_pages - 1)

Explanation: Assigns or updates the value of end_page.

Line 744: page_numbers = list(range(start_page, end_page + 1))

Explanation: Assigns or updates the value of page_numbers.

Line 745: else:

Explanation: Runs the indented block when previous conditions did not match.

Line 746: page_numbers = list(range(1, total_pages + 1))

Explanation: Assigns or updates the value of page_numbers.

Line 747: pages_to_process = len(page_numbers)

Explanation: Assigns or updates the value of pages_to_process.

Line 748: page_range_text = f"{page_numbers[0]} to {page_numbers[-1]}" if page_numbers else "none"

Explanation: Assigns or updates the value of page_range_text.

Line 749: state.setdefault("chapter_page_counts", {})[pdf_stem] = total_pages

Explanation: Assigns or updates the value of state.setdefault("chapter_page_counts", {})[pdf_stem].

Line 750: save_state(config, state)

Explanation: Continues the logic inside process_pdf.

Line 751: [blank]

Explanation: Blank line used to separate logical sections.

Line 752: if total_pages == 0:

Explanation: Starts a conditional branch; the indented block runs only when this condition is true.

Line 753: logging.error("No pages were produced for PDF: %s", pdf_path.name)

Explanation: Writes diagnostic information into the processor log.

Line 754: return

Explanation: Continues the logic inside process_pdf.

Line 755: if config.test_max_pages:

Explanation: Starts a conditional branch; the indented block runs only when this condition is true.

Line 756: logging.info(

Explanation: Writes diagnostic information into the processor log.

Line 757: "Test mode active for %s: processing pages %s of %s total pages.",

Explanation: Continues the logic inside process_pdf.

Line 758: pdf_path.name,

Explanation: Continues the logic inside process_pdf.

Line 759: page_range_text,

Explanation: Continues the logic inside process_pdf.

Line 760: total_pages,

Explanation: Continues the logic inside process_pdf.

Line 761:)

Explanation: Closes a multi-line expression or data structure.

Line 762: print(f"Test mode: processing pages {page_range_text} of {total_pages} total pages.")

Explanation: Writes progress text to the console or job log.

Line 763: [blank]

Explanation: Blank line used to separate logical sections.

Line 764: raw_pdf_dir = config.extracted_dir / pdf_stem

Explanation: Assigns or updates the value of raw_pdf_dir.

Line 765: `verified_pdf_dir = config.verified_dir / pdf_stem`
Explanation: Assigns or updates the value of `verified_pdf_dir`.

Line 766: `raw_pdf_dir.mkdir(parents=True, exist_ok=True)`
Explanation: Creates a folder if it does not already exist.

Line 767: `verified_pdf_dir.mkdir(parents=True, exist_ok=True)`
Explanation: Creates a folder if it does not already exist.

Line 768: `state_lock = threading.Lock()`
Explanation: Assigns or updates the value of `state_lock`.

Line 769: [blank]
Explanation: Blank line used to separate logical sections.

Line 770: `def process_page(index: int) -> None:`
Explanation: Defines the function `process_page`.

Line 771: `verified_path = page_file(verified_pdf_dir, index, "verified")`
Explanation: Assigns or updates the value of `verified_path`.

Line 772: `raw_path = page_file(raw_pdf_dir, index, "raw")`
Explanation: Assigns or updates the value of `raw_path`.

Line 773: [blank]
Explanation: Blank line used to separate logical sections.

Line 774: `if verified_path.exists() and index in set(int_list(state, "verified_pages", pdf_stem)):`
Explanation: Starts a conditional branch; the indented block runs only when this condition is true.

Line 775: `logging.info("Skipping already verified page: %s page %03d", pdf_stem, index)`
Explanation: Writes diagnostic information into the processor log.

Line 776: `return`
Explanation: Continues the logic inside `process_page`.

Line 777: [blank]
Explanation: Blank line used to separate logical sections.

Line 778: `logging.info("Starting page: %s page %03d", pdf_stem, index)`
Explanation: Writes diagnostic information into the processor log.

Line 779: `print(f"Page {index}/{total_pages}")`
Explanation: Writes progress text to the console or job log.

Line 780: [blank]
Explanation: Blank line used to separate logical sections.

Line 781: `try:`
Explanation: Starts a protected block where errors can be caught.

Line 782: `page_client = gemini_client(config)`
Explanation: Assigns or updates the value of `page_client`.

Line 783: `image = convert_pdf_page_to_image(pdf_path, config, index)`
Explanation: Assigns or updates the value of `image`.

Line 784: `if raw_path.exists():`
Explanation: Starts a conditional branch; the indented block runs only when this condition is true.

Line 785: `extracted_text = raw_path.read_text(encoding="utf-8")`
Explanation: Reads text content from a file.

Line 786: `with state_lock:`
Explanation: Opens a context-managed resource and closes it automatically afterward.

Line 787: `mark_page_extracted(config, state, pdf_stem, index)`
Explanation: Continues the logic inside `process_page`.

Line 788: `logging.info("Using existing raw extraction: %s page %03d", pdf_stem, index)`
Explanation: Writes diagnostic information into the processor log.

Line 789: `else:`
Explanation: Runs the indented block when previous conditions did not match.

Line 790: `extracted_text = extract_page_text(page_client, config, state, pdf_stem, index, image)`

Explanation: Assigns or updates the value of `extracted_text`.

Line 791: `raw_path.write_text(extracted_text.strip() + "\n", encoding="utf-8")`

Explanation: Writes text content to a file.

Line 792: `with state_lock:`

Explanation: Opens a context-managed resource and closes it automatically afterward.

Line 793: `mark_page_extracted(config, state, pdf_stem, index)`

Explanation: Continues the logic inside `process_page`.

Line 794: `logging.info("Extraction complete: %s page %03d", pdf_stem, index)`

Explanation: Writes diagnostic information into the processor log.

Line 795: [blank]

Explanation: Blank line used to separate logical sections.

Line 796: `if config.speed_mode == "fast":`

Explanation: Starts a conditional branch; the indented block runs only when this condition is true.

Line 797: `verified_text = extracted_text`

Explanation: Assigns or updates the value of `verified_text`.

Line 798: `logging.info("Fast mode: skipped second-pass verification for %s page %03d", pdf_stem, index)`

Explanation: Writes diagnostic information into the processor log.

Line 799: `else:`

Explanation: Runs the indented block when previous conditions did not match.

Line 800: `verified_text = verify_page_text(page_client, config, state, pdf_stem, index, image, extracted_text)`

Explanation: Assigns or updates the value of `verified_text`.

Line 801: `verified_path.write_text(verified_text.strip() + "\n", encoding="utf-8")`

Explanation: Writes text content to a file.

Line 802: `with state_lock:`

Explanation: Opens a context-managed resource and closes it automatically afterward.

Line 803: `mark_page_verified(config, state, pdf_stem, index)`

Explanation: Continues the logic inside `process_page`.

Line 804: `logging.info("Verification complete: %s page %03d", pdf_stem, index)`

Explanation: Writes diagnostic information into the processor log.

Line 805: `except KeyboardInterrupt:`

Explanation: Handles an error raised inside the matching try block.

Line 806: `raise`

Explanation: Raises an error so the caller knows processing cannot continue normally.

Line 807: `except Exception as exc:`

Explanation: Handles an error raised inside the matching try block.

Line 808: `logging.error("Failed %s page %03d:\n%s", pdf_stem, index, traceback.format_exc())`

Explanation: Writes diagnostic information into the processor log.

Line 809: `with state_lock:`

Explanation: Opens a context-managed resource and closes it automatically afterward.

Line 810: `mark_page_failed(config, state, pdf_stem, index, "page_processing", exc)`

Explanation: Continues the logic inside `process_page`.

Line 811: `print(f"Failed page {index}. Progress was saved; check logs/processor.log and processing_state.json.")`

Explanation: Writes progress text to the console or job log.

Line 812: [blank]

Explanation: Blank line used to separate logical sections.

Line 813: `workers = min(config.page_workers_per_job, len(page_numbers))`

Explanation: Assigns or updates the value of `workers`.

Line 814: if workers <= 1:

Explanation: Starts a conditional branch; the indented block runs only when this condition is true.

Line 815: for index in page_numbers:

Explanation: Starts a loop over each item in a collection.

Line 816: process_page(index)

Explanation: Continues the logic inside process_page.

Line 817: else:

Explanation: Runs the indented block when previous conditions did not match.

Line 818: logging.info("Processing %s with %s page workers in %s mode.", pdf_path.name, workers, config.speed_mode)

Explanation: Writes diagnostic information into the processor log.

Line 819: with ThreadPoolExecutor(max_workers=workers) as executor:

Explanation: Opens a context-managed resource and closes it automatically afterward.

Line 820: futures = [executor.submit(process_page, index) for index in page_numbers]

Explanation: Assigns or updates the value of futures.

Line 821: for future in as_completed(futures):

Explanation: Starts a loop over each item in a collection.

Line 822: future.result()

Explanation: Continues the logic inside process_page.

Line 823: [blank]

Explanation: Blank line used to separate logical sections.

Line 824: if not is_chapter_fully_verified(state, pdf_stem, page_numbers):

Explanation: Starts a conditional branch; the indented block runs only when this condition is true.

Line 825: logging.warning("PDF has unverified failed pages and will not be formatted yet: %s", pdf_path.name)

Explanation: Writes diagnostic information into the processor log.

Line 826: print(f"Skipping chapter formatting for {pdf_path.name} because some pages failed.")

Explanation: Writes progress text to the console or job log.

Line 827: return

Explanation: Continues the logic inside process_page.

Line 828: [blank]

Explanation: Blank line used to separate logical sections.

Line 829: try:

Explanation: Starts a protected block where errors can be caught.

Line 830: output_stem = f"{pdf_stem}_TEST_pages_{page_numbers[0]}_to_{page_numbers[-1]}" if config.test_max_pages
else pdf_stem

Explanation: Assigns or updates the value of output_stem.

Line 831: source_path = combine_verified_pages(config, pdf_stem, page_numbers, output_stem=output_stem)

Explanation: Assigns or updates the value of source_path.

Line 832: logging.info("Verified source created: %s", source_path)

Explanation: Writes diagnostic information into the processor log.

Line 833: [blank]

Explanation: Blank line used to separate logical sections.

Line 834: exact_output_stem = (

Explanation: Assigns or updates the value of exact_output_stem.

Line 835: f"{pdf_stem}_TEST_pages_{page_numbers[0]}_to_{page_numbers[-1]}_exact_verified_transcription"

Explanation: Continues the logic inside process_page.

Line 836: if config.test_max_pages

Explanation: Starts a conditional branch; the indented block runs only when this condition is true.

Line 837: else f"{pdf_stem}_exact_verified_transcription"

Explanation: Continues the logic inside process_page.

Line 838:)
Explanation: Closes a multi-line expression or data structure.

Line 839: verified_docx_path = create_docx_optional(
Explanation: Assigns or updates the value of verified_docx_path.

Line 840: config,
Explanation: Continues the logic inside process_page.

Line 841: pdf_stem,
Explanation: Continues the logic inside process_page.

Line 842: page_numbers,
Explanation: Continues the logic inside process_page.

Line 843: output_stem=exact_output_stem,
Explanation: Assigns or updates the value of output_stem.

Line 844:)
Explanation: Closes a multi-line expression or data structure.

Line 845: [blank]
Explanation: Blank line used to separate logical sections.

Line 846: notes_path: Path | None = None
Explanation: Continues the logic inside process_page.

Line 847: docx_notes_path: Path | None = None
Explanation: Continues the logic inside process_page.

Line 848: final_notes = source_path.read_text(encoding="utf-8")
Explanation: Reads text content from a file.

Line 849: if config.create_structured_notes:
Explanation: Starts a conditional branch; the indented block runs only when this condition is true.

Line 850: formatted_chapters = set(state.setdefault("formatted_chapters", []))
Explanation: Assigns or updates the value of formatted_chapters.

Line 851: notes_path = config.output_dir / f"{output_stem}_nidan_panchak_notes.md"
Explanation: Assigns or updates the value of notes_path.

Line 852: if config.test_max_pages and notes_path.exists():
Explanation: Starts a conditional branch; the indented block runs only when this condition is true.

Line 853: logging.info("Skipping existing test formatted notes: %s", notes_path)
Explanation: Writes diagnostic information into the processor log.

Line 854: elif not config.test_max_pages and pdf_stem in formatted_chapters and notes_path.exists():
Explanation: Starts another conditional branch if the previous conditions were false.

Line 855: logging.info("Skipping already formatted chapter: %s", pdf_stem)
Explanation: Writes diagnostic information into the processor log.

Line 856: else:
Explanation: Runs the indented block when previous conditions did not match.

Line 857: notes = format_chapter_notes(client, config, state, pdf_path, source_path, page_range_text)
Explanation: Assigns or updates the value of notes.

Line 858: notes_path.write_text(notes.strip() + "\n", encoding="utf-8")
Explanation: Writes text content to a file.

Line 859: if not config.test_max_pages:
Explanation: Starts a conditional branch; the indented block runs only when this condition is true.

Line 860: formatted_chapters.add(pdf_stem)
Explanation: Continues the logic inside process_page.

Line 861: state["formatted_chapters"] = sorted(formatted_chapters)
Explanation: Assigns or updates the value of state["formatted_chapters"].

Line 862: save_state(config, state)
Explanation: Continues the logic inside process_page.

Line 863: `logging.info("Formatting complete: %s", pdf_path.name)`

Explanation: Writes diagnostic information into the processor log.

Line 864: [blank]

Explanation: Blank line used to separate logical sections.

Line 865: `final_notes = notes_path.read_text(encoding="utf-8")`

Explanation: Reads text content from a file.

Line 866: `docx_notes_path = create_formatted_notes_docx(config, notes_path)`

Explanation: Assigns or updates the value of docx_notes_path.

Line 867: `else:`

Explanation: Runs the indented block when previous conditions did not match.

Line 868: `logging.info("CREATE_STRUCTURED_NOTES=false, skipped fixed-template notes for %s.", pdf_path.name)`

Explanation: Writes diagnostic information into the processor log.

Line 869: [blank]

Explanation: Blank line used to separate logical sections.

Line 870: `if config.test_max_pages:`

Explanation: Starts a conditional branch; the indented block runs only when this condition is true.

Line 871: `logging.info("Test mode active, not appending partial notes to master and not marking PDF complete.")`

Explanation: Writes diagnostic information into the processor log.

Line 872: `print(f"Verified source created: {source_path}")`

Explanation: Writes progress text to the console or job log.

Line 873: `if docx_notes_path:`

Explanation: Starts a conditional branch; the indented block runs only when this condition is true.

Line 874: `print(f"Test Word document created: {docx_notes_path}")`

Explanation: Writes progress text to the console or job log.

Line 875: `if verified_docx_path:`

Explanation: Starts a conditional branch; the indented block runs only when this condition is true.

Line 876: `print(f"Exact verified transcription Word document created: {verified_docx_path}")`

Explanation: Writes progress text to the console or job log.

Line 877: `print("Partial test notes were not appended to Complete_Book_Notes.md.")`

Explanation: Writes progress text to the console or job log.

Line 878: `else:`

Explanation: Runs the indented block when previous conditions did not match.

Line 879: `append_to_master_notes(config, state, pdf_path.name, final_notes)`

Explanation: Continues the logic inside process_page.

Line 880: `mark_pdf_complete(config, state, pdf_stem)`

Explanation: Continues the logic inside process_page.

Line 881: `except KeyboardInterrupt:`

Explanation: Handles an error raised inside the matching try block.

Line 882: `raise`

Explanation: Raises an error so the caller knows processing cannot continue normally.

Line 883: `except Exception as exc:`

Explanation: Handles an error raised inside the matching try block.

Line 884: `logging.error("Chapter finalization failed for %s:\n%s", pdf_path.name, traceback.format_exc())`

Explanation: Writes diagnostic information into the processor log.

Line 885: `state.setdefault("failed_pages", {}).setdefault(pdf_stem, {})[ "chapter_finalization" ] = {`

Explanation: Assigns or updates the value of state.setdefault("failed_pages", {}).setdefault(pdf_stem, {})["chapter_finalization"].

Line 886: `"stage": "chapter_finalization",`

Explanation: Continues the logic inside process_page.

Line 887: `"error": str(exc),`

Explanation: Continues the logic inside process_page.

Line 888: "timestamp": utc_now(),
Explanation: Continues the logic inside process_page.

Line 889: }
Explanation: Closes a multi-line expression or data structure.

Line 890: save_state(config, state)
Explanation: Continues the logic inside process_page.

Line 891: print(f"Chapter finalization failed for {pdf_path.name}. Progress was saved.")
Explanation: Writes progress text to the console or job log.

Line 892: [blank]
Explanation: Blank line used to separate logical sections.

Line 893: [blank]
Explanation: Blank line used to separate logical sections.

Line 894: def print_summary(config: Config, state: dict[str, Any]) -> None:
Explanation: Prints and logs a final processing summary.

Line 895: completed_pdfs = len(state.get("completed_pdfs", []))
Explanation: Assigns or updates the value of completed_pdfs.

Line 896: failed_pages = sum(
Explanation: Assigns or updates the value of failed_pages.

Line 897: len(pages) for pages in state.get("failed_pages", {}).values() if isinstance(pages, dict)
Explanation: Continues the logic inside print_summary.

Line 898:)
Explanation: Closes a multi-line expression or data structure.

Line 899: logging.info("Final summary: completed_pdfs=%s failed_entries=%s", completed_pdfs, failed_pages)
Explanation: Writes diagnostic information into the processor log.

Line 900: print("\nSummary")
Explanation: Writes progress text to the console or job log.

Line 901: print(f"Completed PDFs: {completed_pdfs}")
Explanation: Writes progress text to the console or job log.

Line 902: print(f"Failed page/finalization entries: {failed_pages}")
Explanation: Writes progress text to the console or job log.

Line 903: print(f"Master notes: {config.master_notes_file}")
Explanation: Writes progress text to the console or job log.

Line 904: print(f"State file: {config.state_file}")
Explanation: Writes progress text to the console or job log.

Line 905: print(f"Log file: {config.logs_dir / 'processor.log'}")
Explanation: Writes progress text to the console or job log.

Line 906: [blank]
Explanation: Blank line used to separate logical sections.

Line 907: [blank]
Explanation: Blank line used to separate logical sections.

Line 908: def main() -> None:
Explanation: Entry point that loads config, state, PDFs, and processes each PDF.

Line 909: config = load_config()
Explanation: Assigns or updates the value of config.

Line 910: setup_logging(config)
Explanation: Continues the logic inside main.

Line 911: ensure_output_dirs(config)
Explanation: Continues the logic inside main.

Line 912: print("Pipeline: Gemini multimodal transcription + verification for scanned textbook pages.")
Explanation: Writes progress text to the console or job log.

Line 913: `logging.info("Pipeline started: Gemini multimodal transcription + verification.")`
Explanation: Writes diagnostic information into the processor log.

Line 914: [blank]
Explanation: Blank line used to separate logical sections.

Line 915: `if not config.gemini_api_key:`
Explanation: Starts a conditional branch; the indented block runs only when this condition is true.

Line 916: `logging.error("Missing GEMINI_API_KEY. Add it to .env and run again.")`
Explanation: Writes diagnostic information into the processor log.

Line 917: `print("Missing GEMINI_API_KEY. Create/edit .env and set GEMINI_API_KEY=your_key_here.")`
Explanation: Writes progress text to the console or job log.

Line 918: `return`
Explanation: Continues the logic inside main.

Line 919: [blank]
Explanation: Blank line used to separate logical sections.

Line 920: `state = load_state(config)`
Explanation: Assigns or updates the value of state.

Line 921: `if config.reset_master and not config.test_max_pages:`
Explanation: Starts a conditional branch; the indented block runs only when this condition is true.

Line 922: `config.master_notes_file.write_text("", encoding="utf-8")`
Explanation: Writes text content to a file.

Line 923: `state["master_appended_chapters"] = []`
Explanation: Assigns or updates the value of state["master_appended_chapters"].

Line 924: `logging.info("RESET_MASTER=true, master notes file was cleared.")`
Explanation: Writes diagnostic information into the processor log.

Line 925: `elif config.reset_master and config.test_max_pages:`
Explanation: Starts another conditional branch if the previous conditions were false.

Line 926: `logging.info("RESET_MASTER=true ignored during test mode to protect master notes.")`
Explanation: Writes diagnostic information into the processor log.

Line 927: `print("RESET_MASTER=true is ignored during test mode to protect Complete_Book_Notes.md.")`
Explanation: Writes progress text to the console or job log.

Line 928: `save_state(config, state)`
Explanation: Continues the logic inside main.

Line 929: `pdfs = find_pdfs(config)`
Explanation: Assigns or updates the value of pdfs.

Line 930: `if not pdfs:`
Explanation: Starts a conditional branch; the indented block runs only when this condition is true.

Line 931: `print(f"No PDF files found in {config.pdf_dir}. Add scanned PDFs and run again.")`
Explanation: Writes progress text to the console or job log.

Line 932: `logging.warning("No PDFs found in %s", config.pdf_dir)`
Explanation: Writes diagnostic information into the processor log.

Line 933: `return`
Explanation: Continues the logic inside main.

Line 934: [blank]
Explanation: Blank line used to separate logical sections.

Line 935: `client = gemini_client(config)`
Explanation: Assigns or updates the value of client.

Line 936: [blank]
Explanation: Blank line used to separate logical sections.

Line 937: `try:`
Explanation: Starts a protected block where errors can be caught.

Line 938: if config.test_pdf_limit:

Explanation: Starts a conditional branch; the indented block runs only when this condition is true.

Line 939: pdfs = pdfs[: config.test_pdf_limit]

Explanation: Assigns or updates the value of pdfs.

Line 940: logging.info("TEST_PDF_LIMIT active: processing first %s PDF(s).", config.test_pdf_limit)

Explanation: Writes diagnostic information into the processor log.

Line 941: print(f"Test PDF limit: processing first {len(pdf)} PDF(s).")

Explanation: Writes progress text to the console or job log.

Line 942: for pdf_path in pdfs:

Explanation: Starts a loop over each item in a collection.

Line 943: process_pdf(client, config, state, pdf_path)

Explanation: Continues the logic inside main.

Line 944: except KeyboardInterrupt:

Explanation: Handles an error raised inside the matching try block.

Line 945: save_state(config, state)

Explanation: Continues the logic inside main.

Line 946: logging.warning("Interrupted by user. Progress saved.")

Explanation: Writes diagnostic information into the processor log.

Line 947: print("\nInterrupted. Progress was saved in processing_state.json.")

Explanation: Writes progress text to the console or job log.

Line 948: except Exception:

Explanation: Handles an error raised inside the matching try block.

Line 949: save_state(config, state)

Explanation: Continues the logic inside main.

Line 950: logging.error("Unexpected top-level error:\n%s", traceback.format_exc())

Explanation: Writes diagnostic information into the processor log.

Line 951: print("Unexpected error. Progress was saved; check logs/processor.log.")

Explanation: Writes progress text to the console or job log.

Line 952: finally:

Explanation: Runs cleanup code whether the try block succeeded or failed.

Line 953: print_summary(config, state)

Explanation: Continues the logic inside main.

Line 954: [blank]

Explanation: Blank line used to separate logical sections.

Line 955: [blank]

Explanation: Blank line used to separate logical sections.

Line 956: if __name__ == "__main__":

Explanation: Starts a conditional branch; the indented block runs only when this condition is true.

Line 957: main()

Explanation: Continues the logic inside main.

start_production.ps1

PowerShell launcher for the production Waitress server.

Source path: C:\Users\sawan\Desktop\new_project\Ayurvedic_Book_Processor\start_production.ps1

Line count: 17

Line 1: \$ErrorActionPreference = "Stop"

Explanation: Makes PowerShell stop immediately if a command fails.

Line 2: [blank]

Explanation: Blank line used to separate script steps.

Line 3: Set-Location \$PSScriptRoot

Explanation: Changes the current folder to the folder containing this script.

Line 4: [blank]

Explanation: Blank line used to separate script steps.

Line 5: if (-not \$env:HOST) { \$env:HOST = "0.0.0.0" }

Explanation: Sets the server host to 0.0.0.0 if HOST was not already provided.

Line 6: if (-not \$env:PORT) { \$env:PORT = "7860" }

Explanation: Sets the server port to 7860 if PORT was not already provided.

Line 7: if (-not \$env:MAX_UPLOAD_MB) { \$env:MAX_UPLOAD_MB = "250" }

Explanation: Sets the default upload limit to 250 MB if not already provided.

Line 8: [blank]

Explanation: Blank line used to separate script steps.

Line 9: # Avoid inherited local proxy placeholders breaking Gemini API calls.

Explanation: Comment explaining the next script action.

Line 10: Remove-Item Env:HTTP_PROXY -ErrorAction SilentlyContinue

Explanation: Removes inherited proxy environment variables that can break Gemini API calls.

Line 11: Remove-Item Env:HTTPS_PROXY -ErrorAction SilentlyContinue

Explanation: Removes inherited proxy environment variables that can break Gemini API calls.

Line 12: Remove-Item Env:ALL_PROXY -ErrorAction SilentlyContinue

Explanation: Removes inherited proxy environment variables that can break Gemini API calls.

Line 13: Remove-Item Env:http_proxy -ErrorAction SilentlyContinue

Explanation: Removes inherited proxy environment variables that can break Gemini API calls.

Line 14: Remove-Item Env:https_proxy -ErrorAction SilentlyContinue

Explanation: Removes inherited proxy environment variables that can break Gemini API calls.

Line 15: Remove-Item Env:all_proxy -ErrorAction SilentlyContinue

Explanation: Removes inherited proxy environment variables that can break Gemini API calls.

Line 16: [blank]

Explanation: Blank line used to separate script steps.

Line 17: .\venv\Scripts\waitress-serve.exe --host=\$env:HOST --port=\$env:PORT app:app

Explanation: Starts the Flask app with Waitress using the configured host and port.

.env.example

Template for production configuration. Real secrets are kept in .env and are not printed here.

Source path: C:\Users\sawan\Desktop\new_project\Ayurvedic_Book_Processor\.env.example

Line count: 21

Line 1: GEMINI_API_KEY=your_key_here

Explanation: Gemini API key used by the processor. The real value belongs only in .env.

Line 2: GEMINI_MODEL=gemini-2.5-flash

Explanation: Gemini model name used for extraction and verification calls.

Line 3: PDF_DIR=./pdfs

Explanation: Folder where the processor looks for PDFs inside each job.

Line 4: OUTPUT_DIR=./output_notes

Explanation: Folder where Markdown and DOCX outputs are written.

Line 5: POPPLER_PATH=C:\poppler\bin

Explanation: Windows path to Poppler binaries used by pdf2image.

Line 6: DPI=250

Explanation: Image resolution for converting PDF pages before sending to Gemini.

Line 7: MAX_RETRIES=5

Explanation: Maximum retry attempts for failed Gemini calls.

Line 8: GEMINI_DELAY_SECONDS=1

Explanation: Delay before Gemini calls to reduce quota pressure.

Line 9: GEMINI_TIMEOUT_SECONDS=180

Explanation: Maximum time one Gemini request may wait before retrying.

Line 10: RESET_MASTER=false

Explanation: Whether to clear the master notes file during full processing.

Line 11: CREATE_DOCX=true

Explanation: Whether to create Word documents.

Line 12: TEST_MAX_PAGES=0

Explanation: Page limit for test runs; zero means process the full PDF.

Line 13: TEST_PDF_LIMIT=1

Explanation: Maximum PDFs to process per job; one is used for uploaded jobs.

Line 14: CREATE_STRUCTURED_NOTES=false

Explanation: Whether to ask Gemini for structured notes in addition to exact transcription.

Line 15: MAX_PARALLEL_JOBS=2

Explanation: Maximum number of job processes running at once.

Line 16: TEST_START_PAGE=1

Explanation: First page to process in test mode.

Line 17: PAGE_WORKERS_PER_JOB=1

Explanation: How many pages inside one PDF may process in parallel.

Line 18: SPEED_MODE=accuracy

Explanation: Controls fast, balanced, or accuracy processing behavior.

Line 19: MAX_UPLOAD_MB=250

Explanation: Maximum upload size accepted by the Flask app.

Line 20: APP_USERNAME=team

Explanation: Username for web login.

Line 21: APP_PASSWORD=change-this-password

Explanation: Password for web login.